

Sorting

Sorting Algorithms

➤ Comparison Sorting

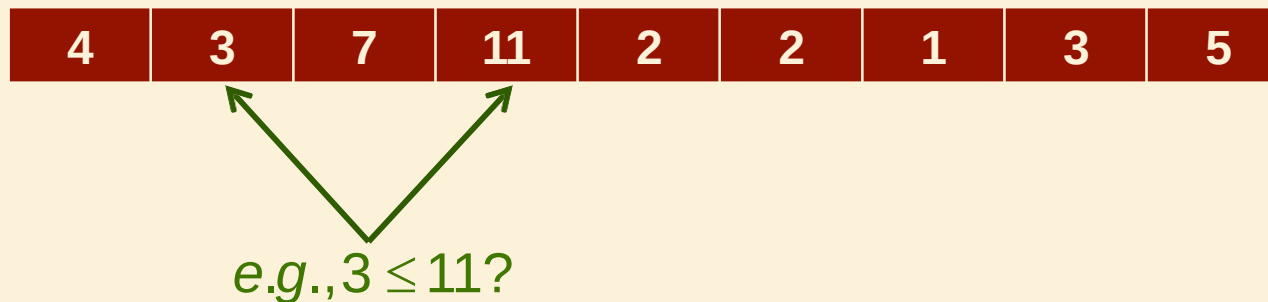
- ☐ Selection Sort
- ☐ Bubble Sort
- ☐ Insertion Sort
- ☐ Merge Sort
- ☐ Heap Sort
- ☐ Quick Sort

➤ Linear Sorting

- ☐ Counting Sort
- ☐ Radix Sort
- ☐ Bucket Sort

Comparison Sorts

- Comparison Sort algorithms sort the input by successive comparison of pairs of input elements.
- Comparison Sort algorithms are very general: they make no assumptions about the values of the input elements.



Sorting Algorithms and Memory

- Some algorithms sort by swapping elements within the input array
- Such algorithms are said to **sort in place**, and require only $O(1)$ additional memory.
- Other algorithms require allocation of an output array into which values are copied.
- These algorithms do not sort in place, and require $O(n)$ additional memory.



Selection Sort

- Selection Sort operates by first finding the smallest element in the input list, and moving it to the output list.
- It then finds the next smallest value and does the same.
- It continues in this way until all the input elements have been selected and placed in the output list in the correct order.
- Note that every selection requires a search through the input list.
- Thus the algorithm has a nested loop structure
- Selection Sort Example

Selection Sort

for $i = n-1$ downto 0

$j_{\min} = 0$

 for $j = 1$ to i

 if $A[j] < A[j_{\min}]$

$j_{\min} = j$

 add $A[j_{\min}]$ to output

 remove $A[j_{\min}]$ from input

} $O(i)$

$$T(n) = \sum_{i=0}^{n-2} i = O(n^2)$$

Bubble Sort

- Bubble Sort operates by successively comparing adjacent elements, swapping them if they are out of order.
- At the end of the first pass, the largest element is in the correct position.
- A total of n passes are required to sort the entire array.
- Thus bubble sort also has a nested loop structure
- Bubble Sort Example

Expert Opinion on Bubble Sort

Bubble Sort

for i = n-2 downto 0

for j = 0 to i

if $A[j] > A[j + 1]$

swap $A[j]$ and $A[j + 1]$

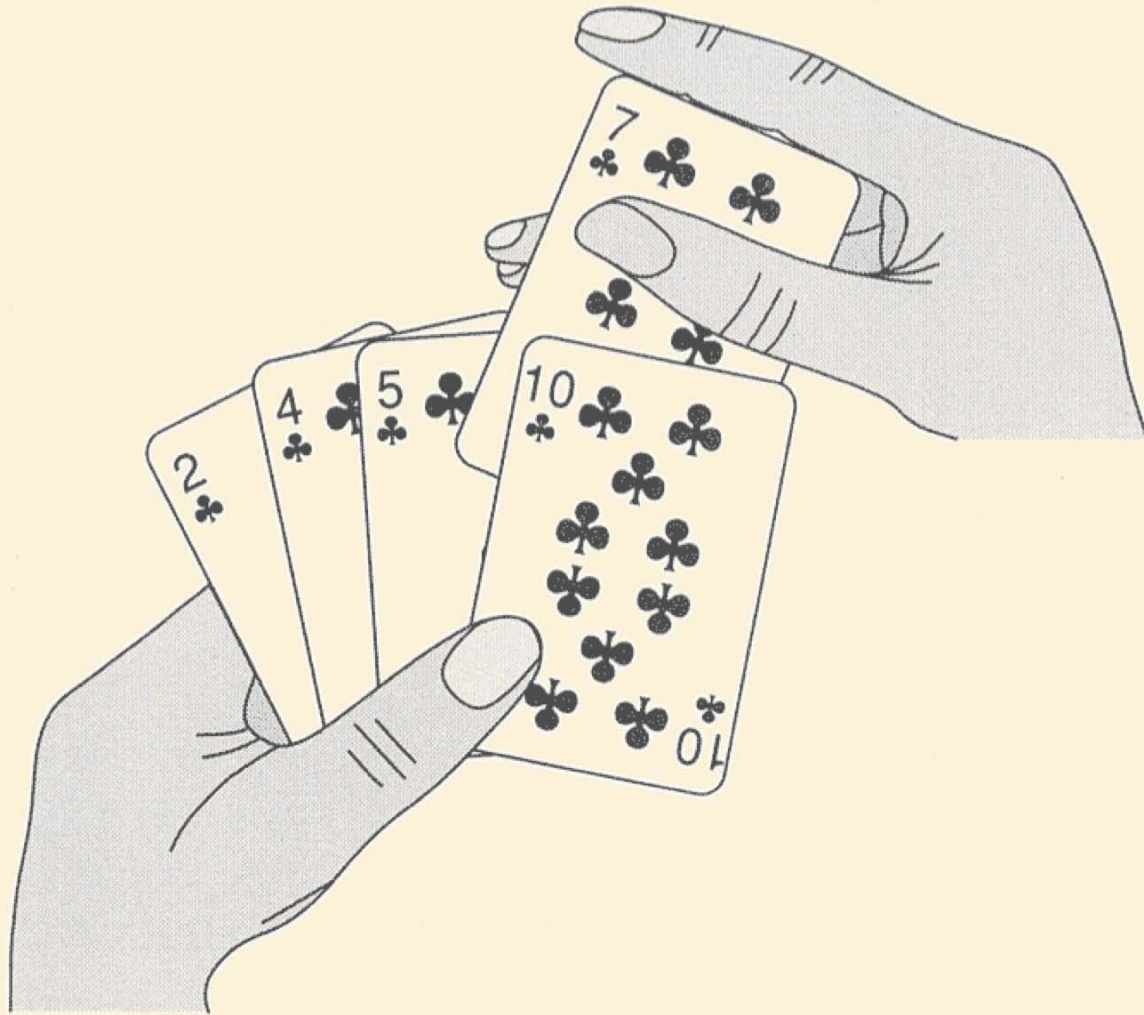
} $O(i)$

$$T(n) = \sum_{i=0}^{n-2} i = O(n^2)$$

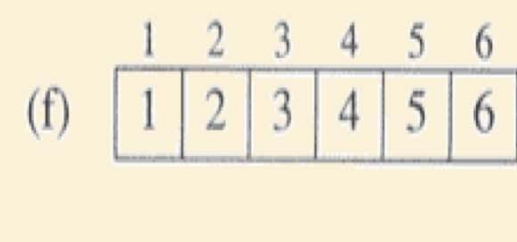
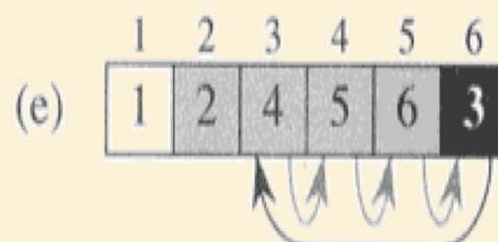
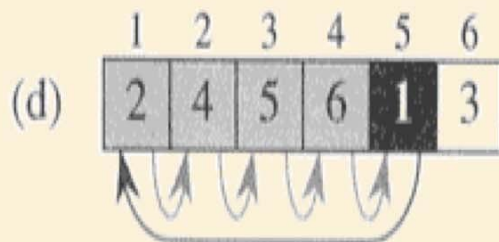
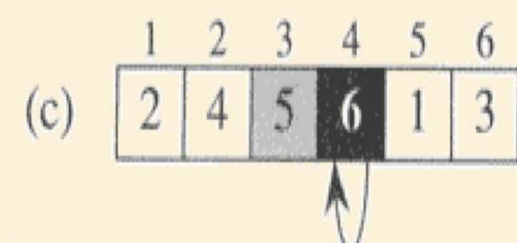
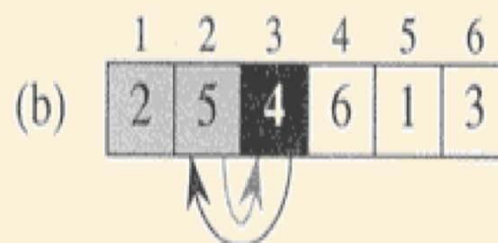
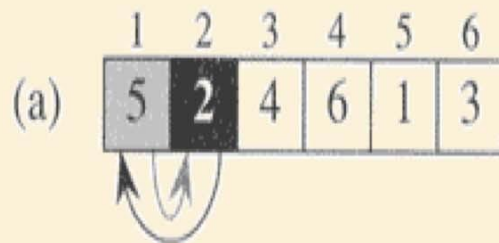
Comparison

- Thus both Selection Sort and Bubble Sort have $O(n^2)$ running time.
- However, both can also easily be designed to
 - ❑ Sort in place

Example: Insertion Sort



Example: Insertion Sort



Example: Insertion Sort

INSERTION-SORT(<i>A</i>)		<i>cost</i>	<i>times</i>
1	for $j \leftarrow 2$ to $\text{length}[A]$	c_1	n
2	do $\text{key} \leftarrow A[j]$	c_2	$n - 1$
3	$\triangleright$ Insert $A[j]$ into the sorted sequence $A[1 \dots j - 1]$.	0	$n - 1$
4	$i \leftarrow j - 1$	c_4	$n - 1$
5	while $i > 0$ and $A[i] > \text{key}$	c_5	$\sum_{j=2}^n t_j$
6	do $A[i + 1] \leftarrow A[i]$	c_6	$\sum_{j=2}^n (t_j - 1)$
7	$i \leftarrow i - 1$	c_7	$\sum_{j=2}^n (t_j - 1)$
8	$A[i + 1] \leftarrow \text{key}$	c_8	$n - 1$

Worst case (reverse order): $t_j = j$: $\sum_{j=2}^n j = \frac{n(n+1)}{2} - 1 \rightarrow T(n) \in \theta(n^2)$

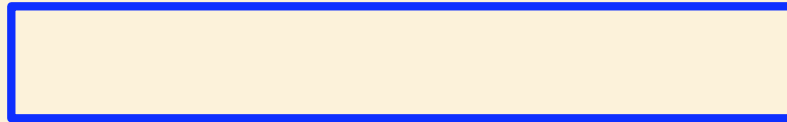
Insertion Sort Example

Comparison

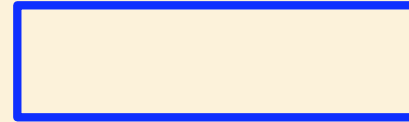
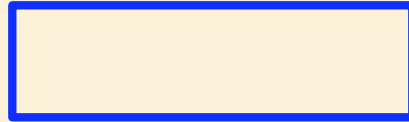
- Selection Sort
- Bubble Sort
- Insertion Sort
 - ❑ Sort in place
 - ❑ But $O(n^2)$ running time.
- Can we do better?

Recursive Sorts

- Given list of objects to be sorted



- Split the list into two sublists.



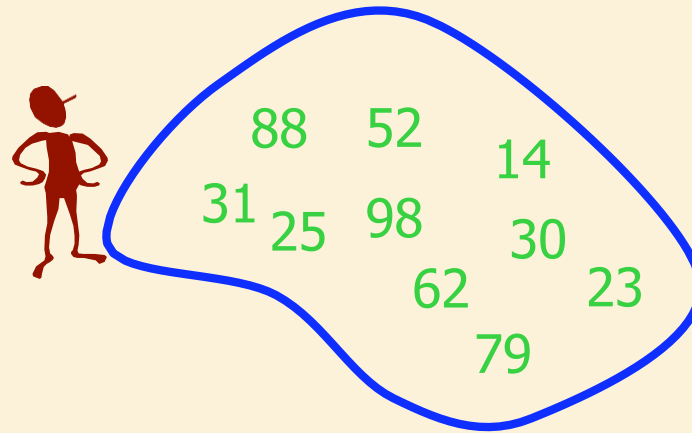
- Recursively have a friend sort the two sublists



- Combine the two sorted sublists into one entirely sorted list.



Merge Sort



Divide and Conquer



Merge Sort

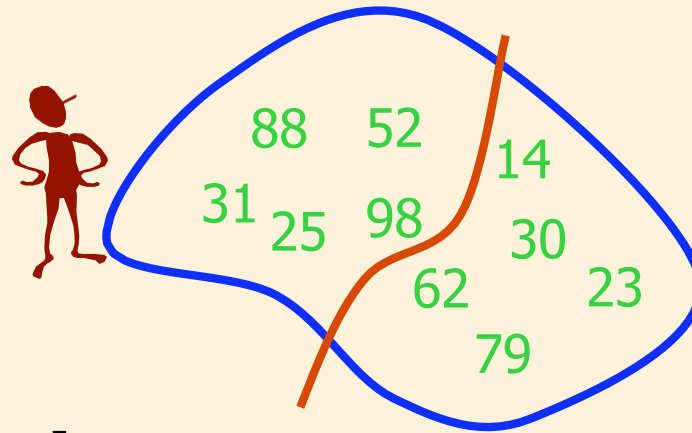
- **Merge-sort** is a sorting algorithm based on the divide-and-conquer paradigm
- It was invented by John von Neumann, one of the pioneers of computing, in 1945



Divide-and-Conquer

- **Divide-and conquer** is a general algorithm design paradigm:
 - ❑ **Divide**: divide the input data S in two disjoint subsets S_1 and S_2
 - ❑ **Recur**: solve the subproblems associated with S_1 and S_2
 - ❑ **Conquer**: combine the solutions for S_1 and S_2 into a solution for S
- The base case for the recursion are subproblems of size 0 or 1

Merge Sort



Split Set into Two
(no real work)

Get one friend to
sort the first half.



25,31,52,88,98

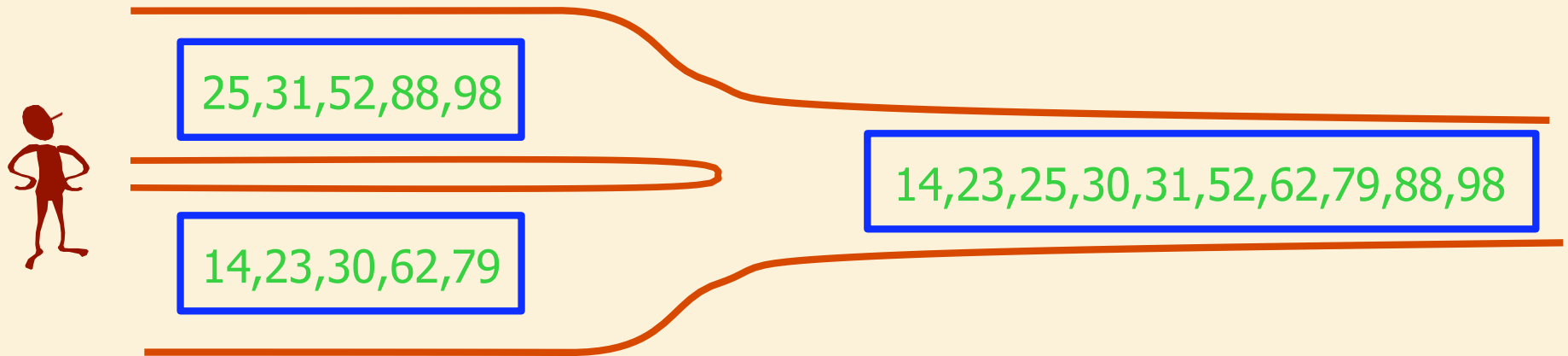
Get one friend to
sort the second half.



14,23,30,62,79

Merge Sort

Merge two sorted lists into one



Merge-Sort

- Merge-sort on an input sequence S with n elements consists of three steps:
 - ❑ **Divide**: partition S into two sequences S_1 and S_2 of about $n/2$ elements each
 - ❑ **Recur**: recursively sort S_1 and S_2
 - ❑ **Conquer**: merge S_1 and S_2 into a unique sorted sequence

Algorithm *mergeSort(S)*

Input sequence S with n elements

Output sequence S sorted

if $S.size() > 1$

$(S_1, S_2) \leftarrow split(S, n/2)$

mergeSort(S_1)

mergeSort(S_2)

merge(S_1, S_2, S)

Merging Two Sorted Sequences

- The conquer step of merge-sort consists of merging two sorted sequences **A** and **B** into a sorted sequence **S** containing the union of the elements of **A** and **B**
- Merging two sorted sequences, each with $n/2$ elements takes $O(n)$ time
- Normally, merging is not in-place: new memory must be allocated to hold S.
- It **is** possible to do in-place merging using linked lists.
 - ❑ Code is more complicated
 - ❑ Only changes memory usage by a constant factor

Merging Two Sorted Sequences (As Arrays)

Algorithm merge(S_1, S_2, S):

Input: Sorted sequences S_1 and S_2 and an empty sequence S , implemented as arrays

Output: Sorted sequence S containing the elements from S_1 and S_2

$i \leftarrow j \leftarrow 0$

while $i < S_1.size()$ **and** $j < S_2.size()$ **do**

if $S_1.get(i) \leq S_2.get(j)$ **then**

$S.addLast(S_1.get(i))$

$i \leftarrow i + 1$

else

$S.addLast(S_2.get(j))$

$j \leftarrow j + 1$

while $i < S_1.size()$ **do**

$S.addLast(S_1.get(i))$

$i \leftarrow i + 1$

while $j < S_2.size()$ **do**

$S.addLast(S_2.get(j))$

$j \leftarrow j + 1$

Merging Two Sorted Sequences (As Linked Lists)

Algorithm merge(S_1 , S_2 , S):

Input: Sorted sequences S_1 and S_2 and an empty sequence S , implemented as linked lists

Output: Sorted sequence S containing the elements from S_1 and S_2

while $S_1 \neq \text{null}$ **and** $S_2 \neq \text{null}$ **do**

if $S_1.\text{first}().\text{element}() \leq S_2.\text{first}().\text{element}()$ **then**

$S.\text{addLast}(S_1.\text{remove}(S_1.\text{first}()))$

$i \leftarrow i + 1$

else

$S.\text{addLast}(S_2.\text{remove}(S_2.\text{first}()))$

while $S_1 \neq \text{null}$ **do**

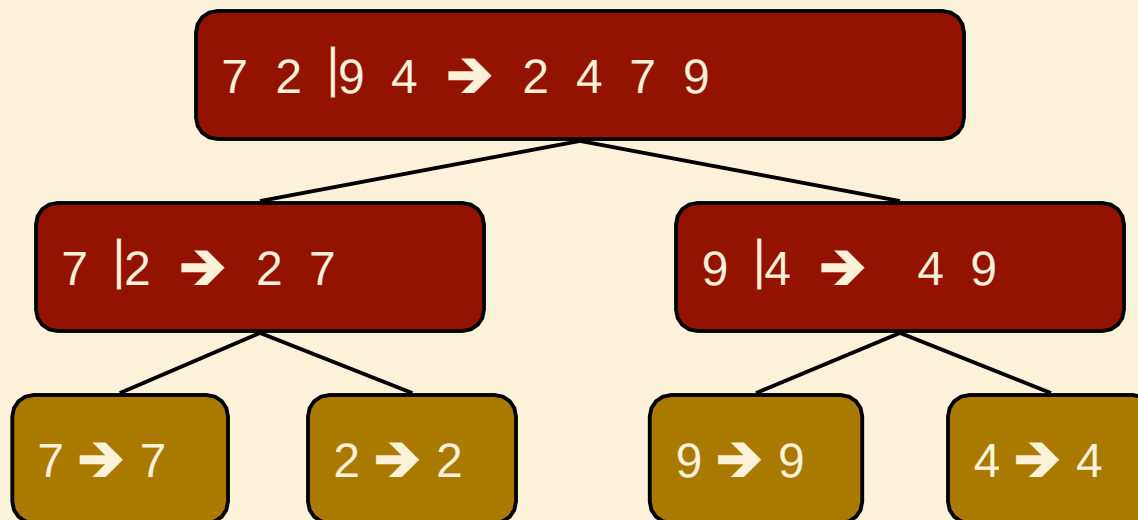
$S.\text{addLast}(S_1.\text{remove}(S_1.\text{first}()))$

while $S_2 \neq \text{null}$ **do**

$S.\text{addLast}(S_2.\text{remove}(S_2.\text{first}()))$

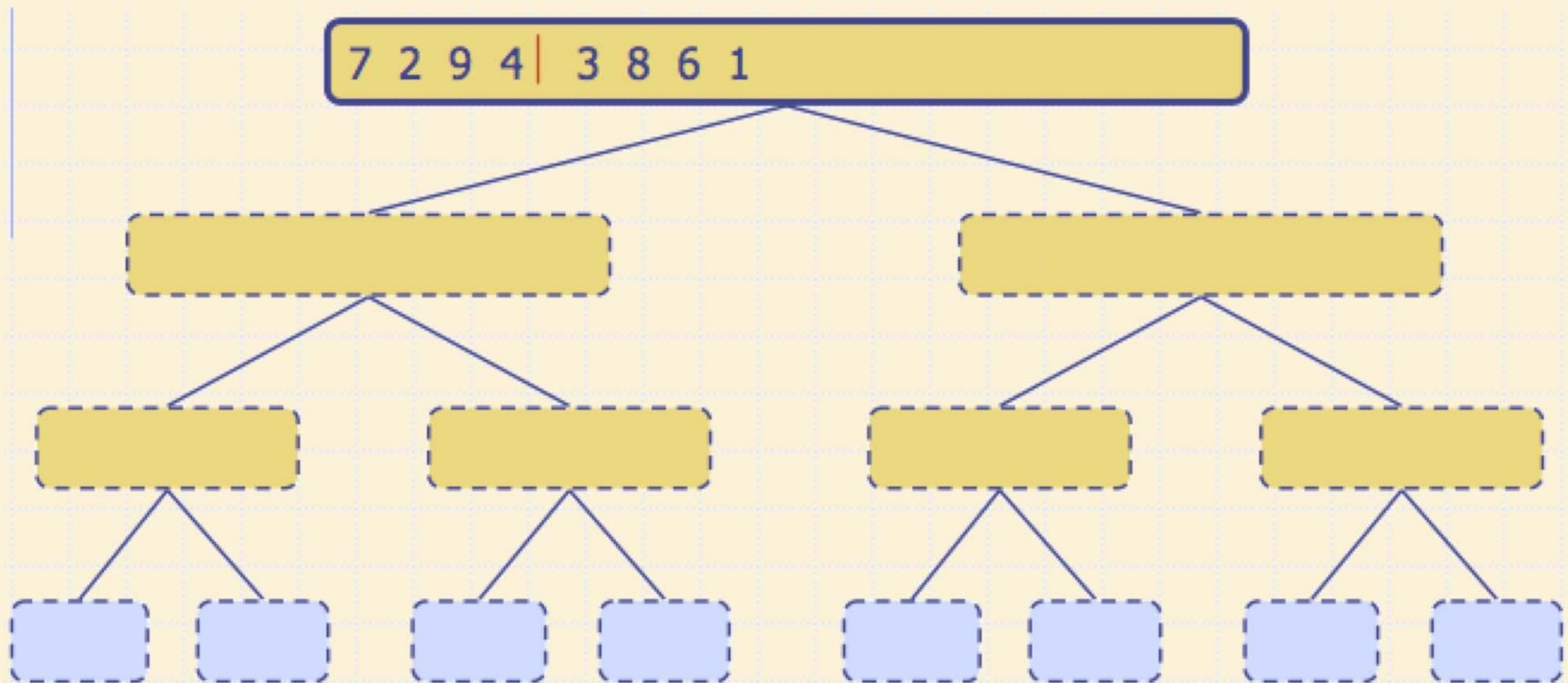
Merge-Sort Tree

- An execution of merge-sort is depicted by a binary tree
 - ❑ each node represents a recursive call of merge-sort and stores
 - ✧ unsorted sequence before the execution and its partition
 - ✧ sorted sequence at the end of the execution
 - ❑ the root is the initial call
 - ❑ the leaves are calls on subsequences of size 0 or 1



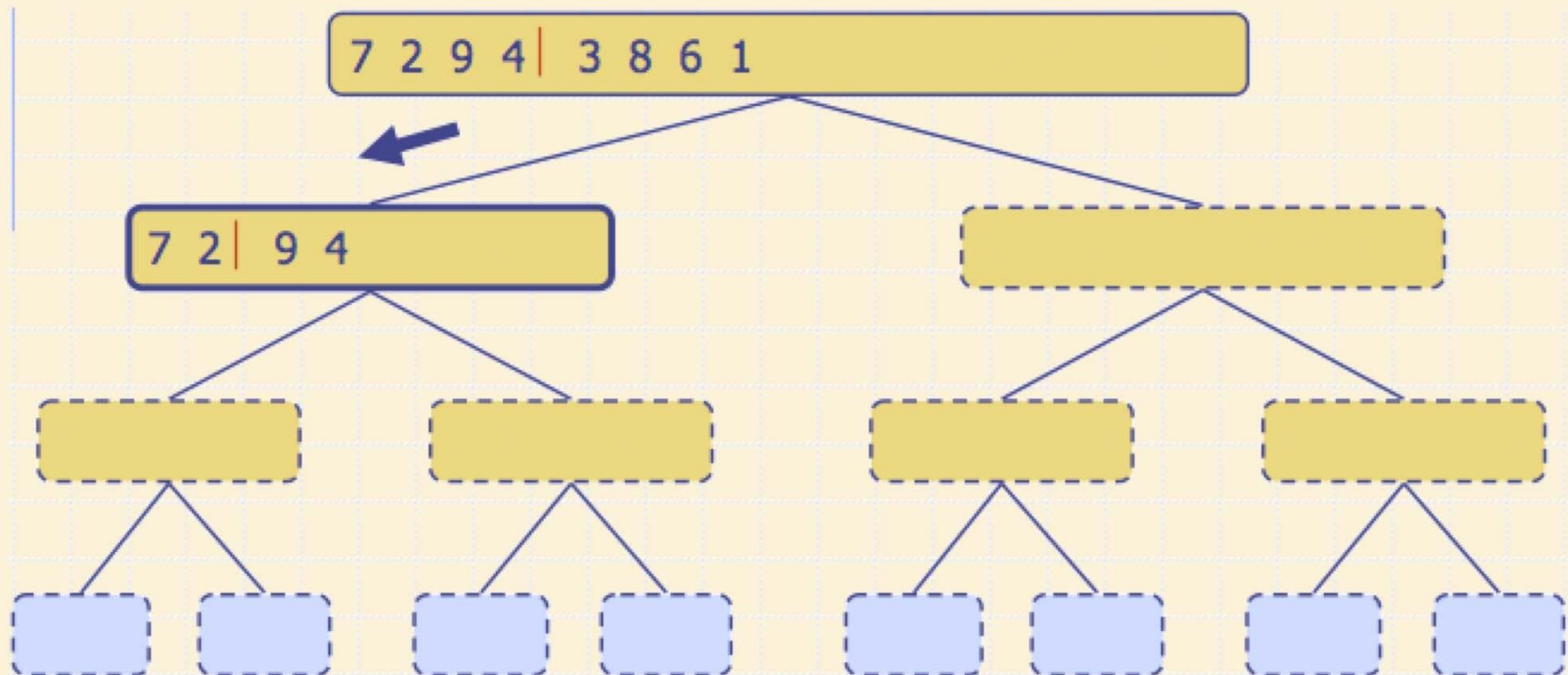
Execution Example

➤ Partition



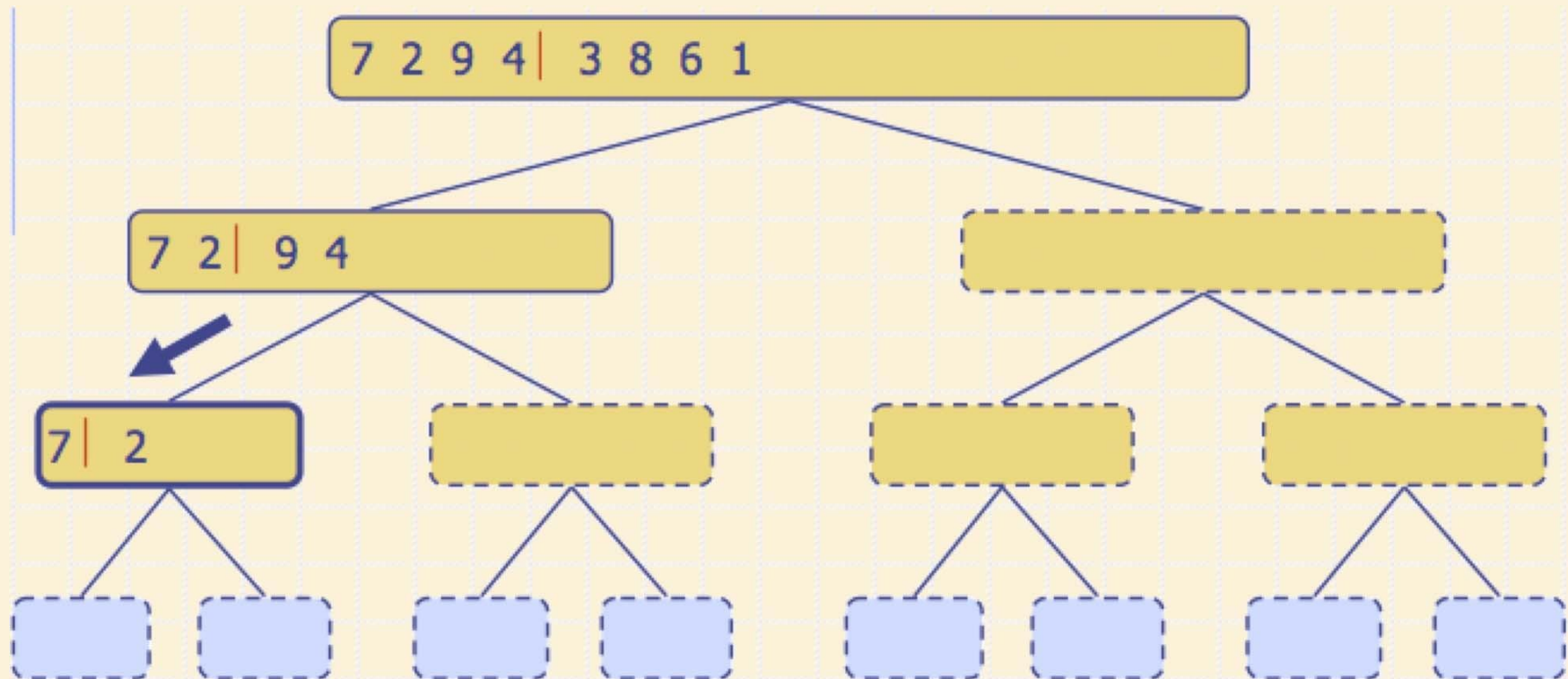
Execution Example (cont.)

- Recursive call, partition



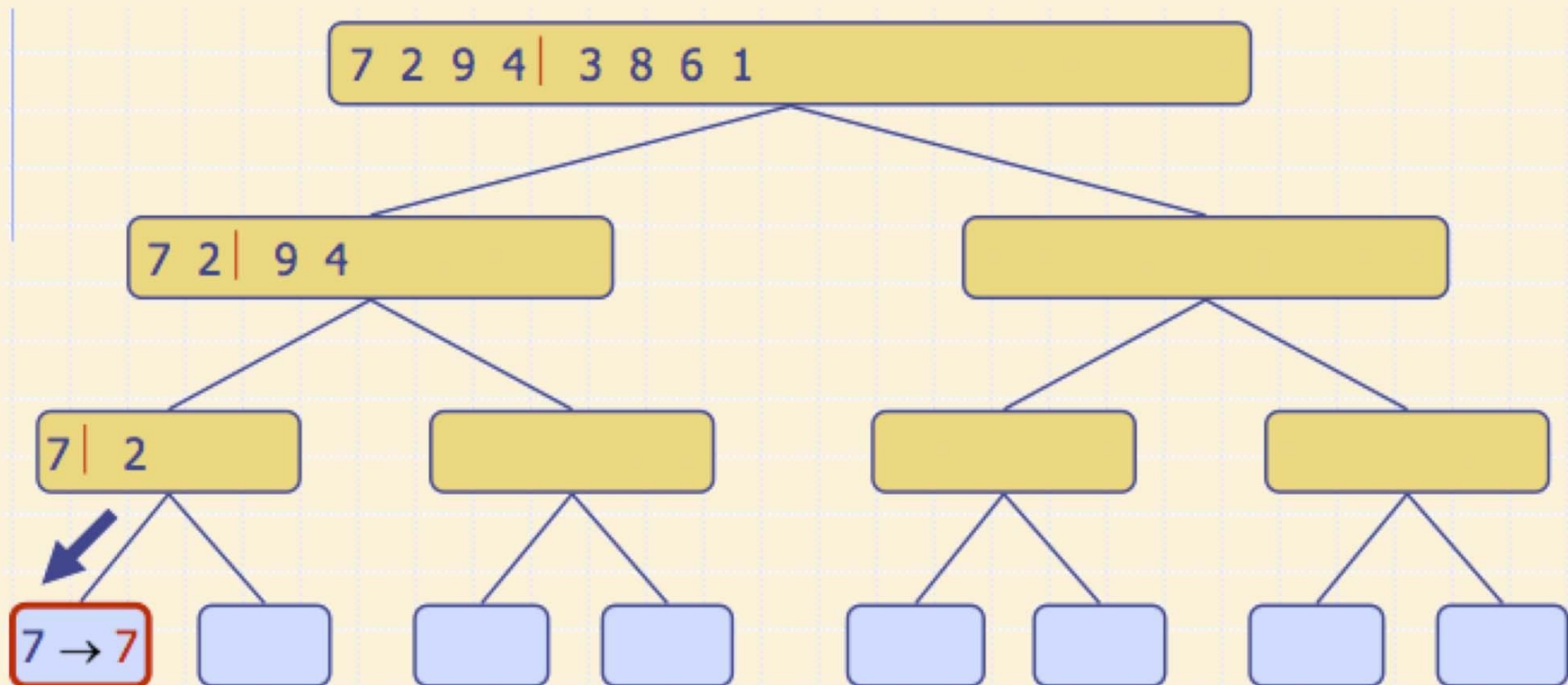
Execution Example (cont.)

- Recursive call, partition



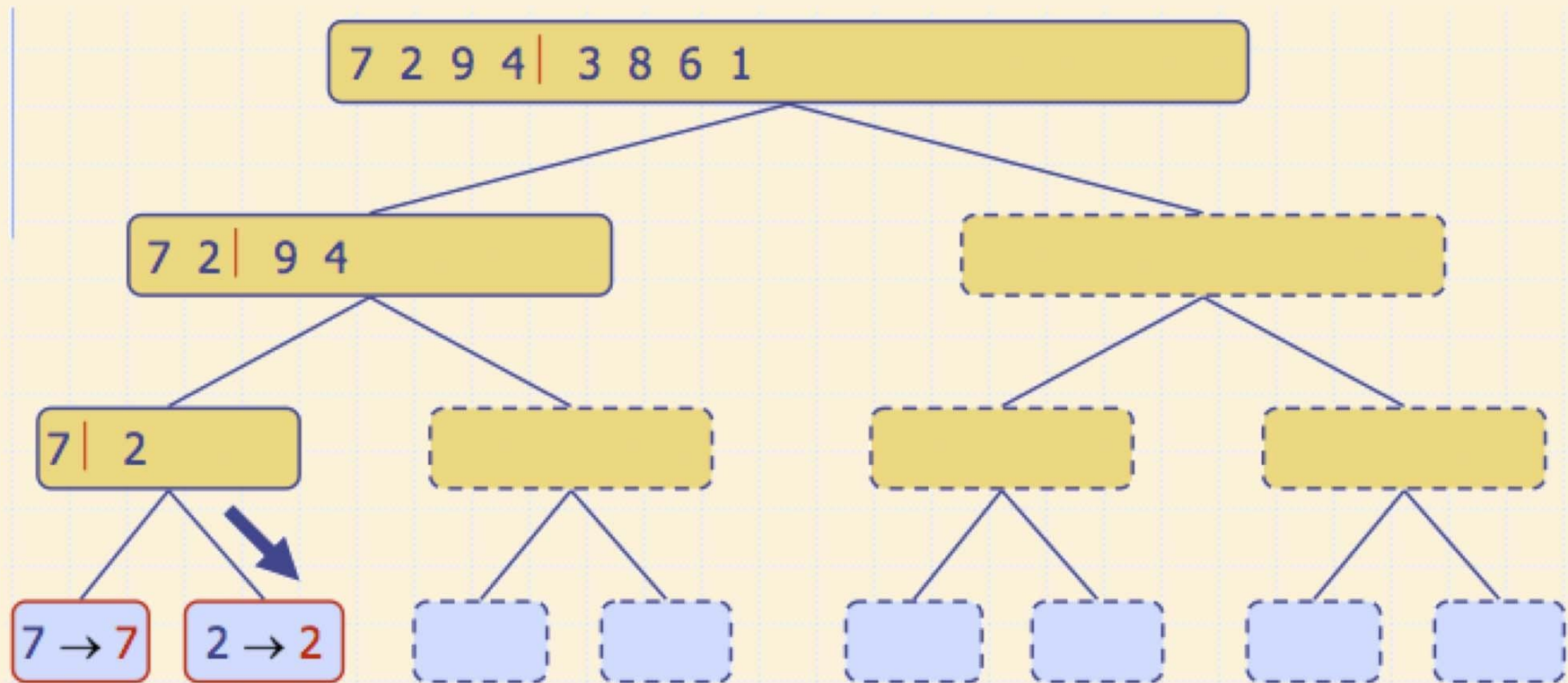
Execution Example (cont.)

- Recursive call, base case



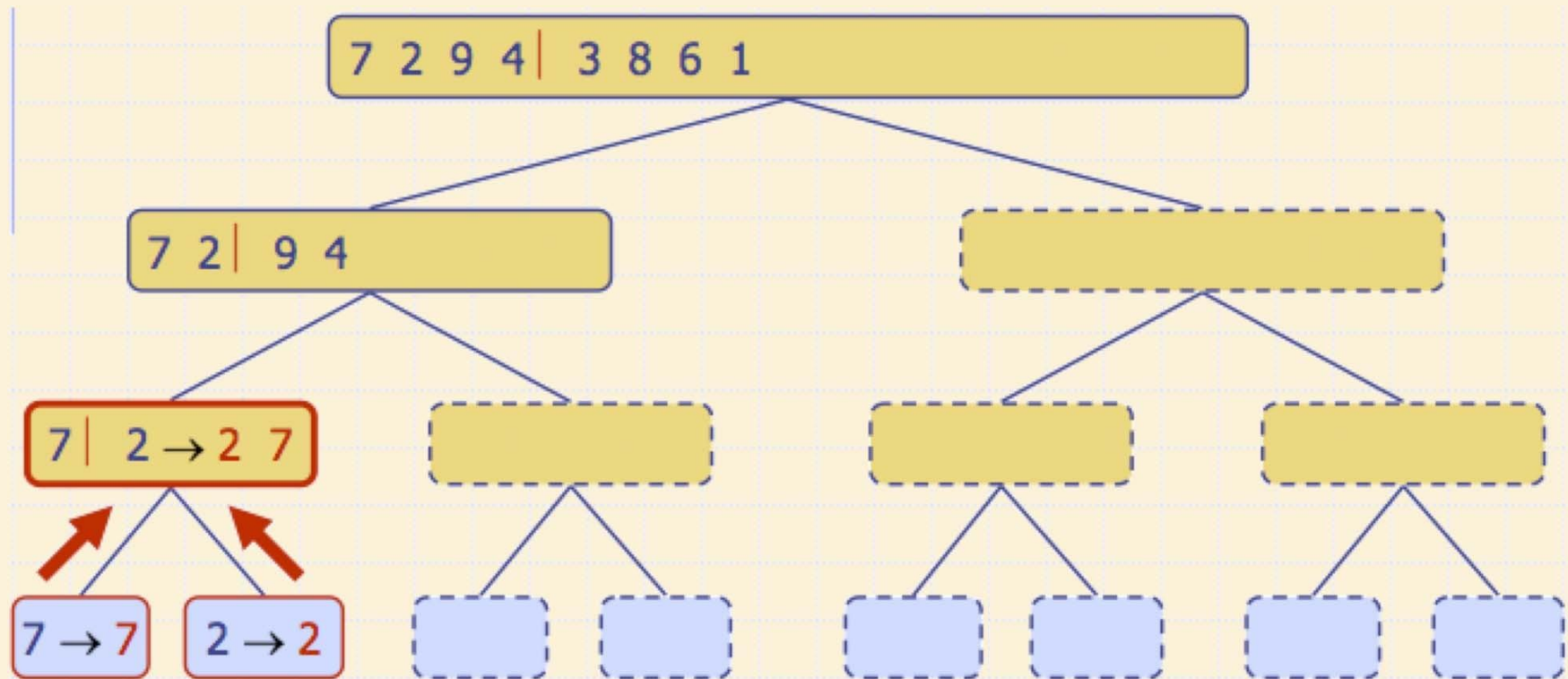
Execution Example (cont.)

- Recursive call, base case



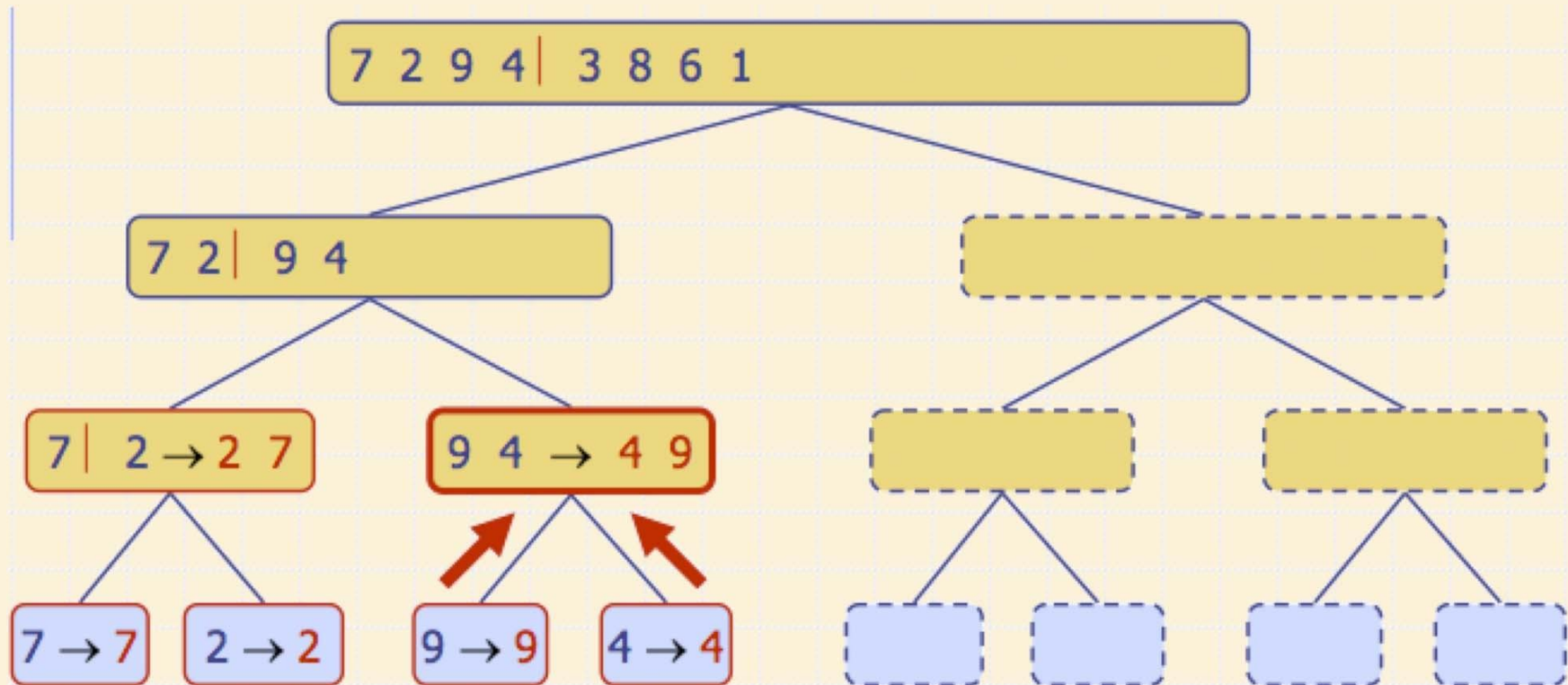
Execution Example (cont.)

➤ Merge



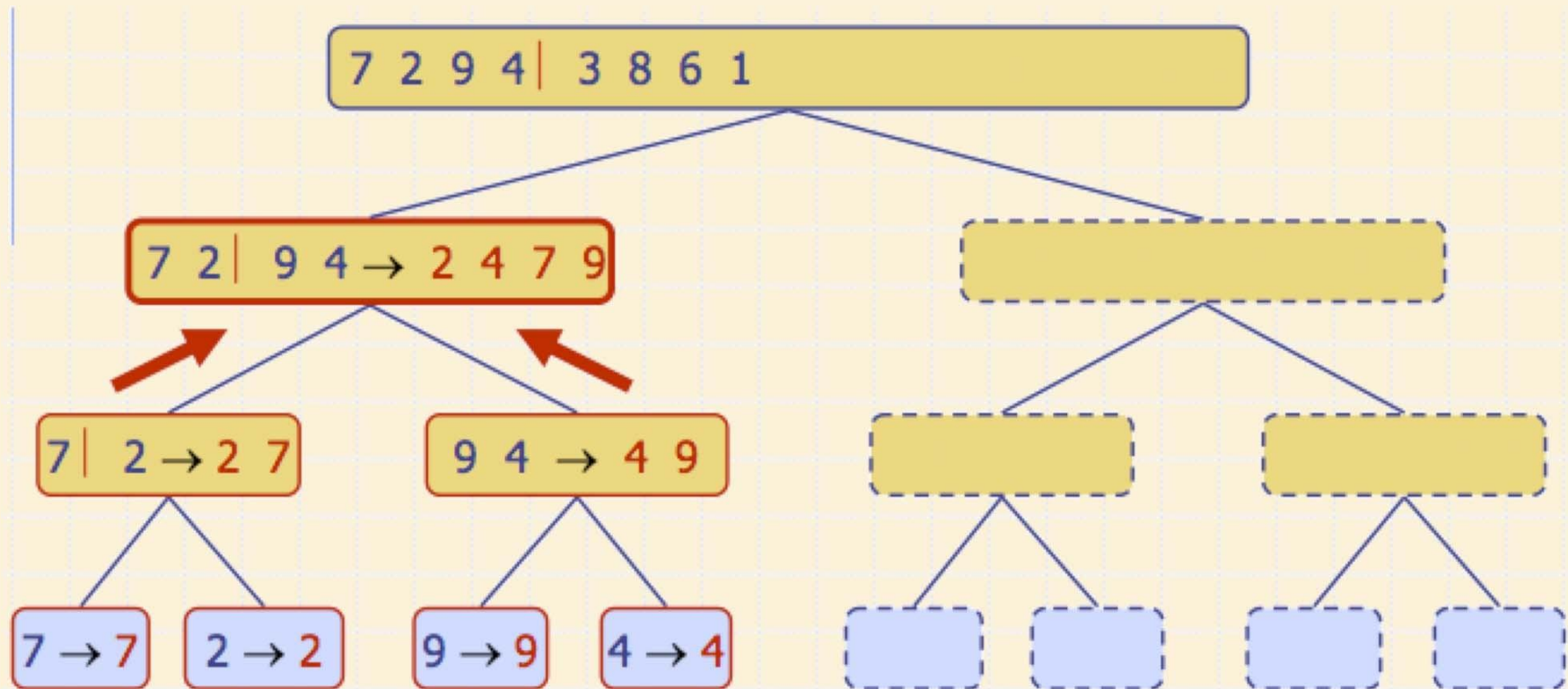
Execution Example (cont.)

- Recursive call, ..., base case, merge



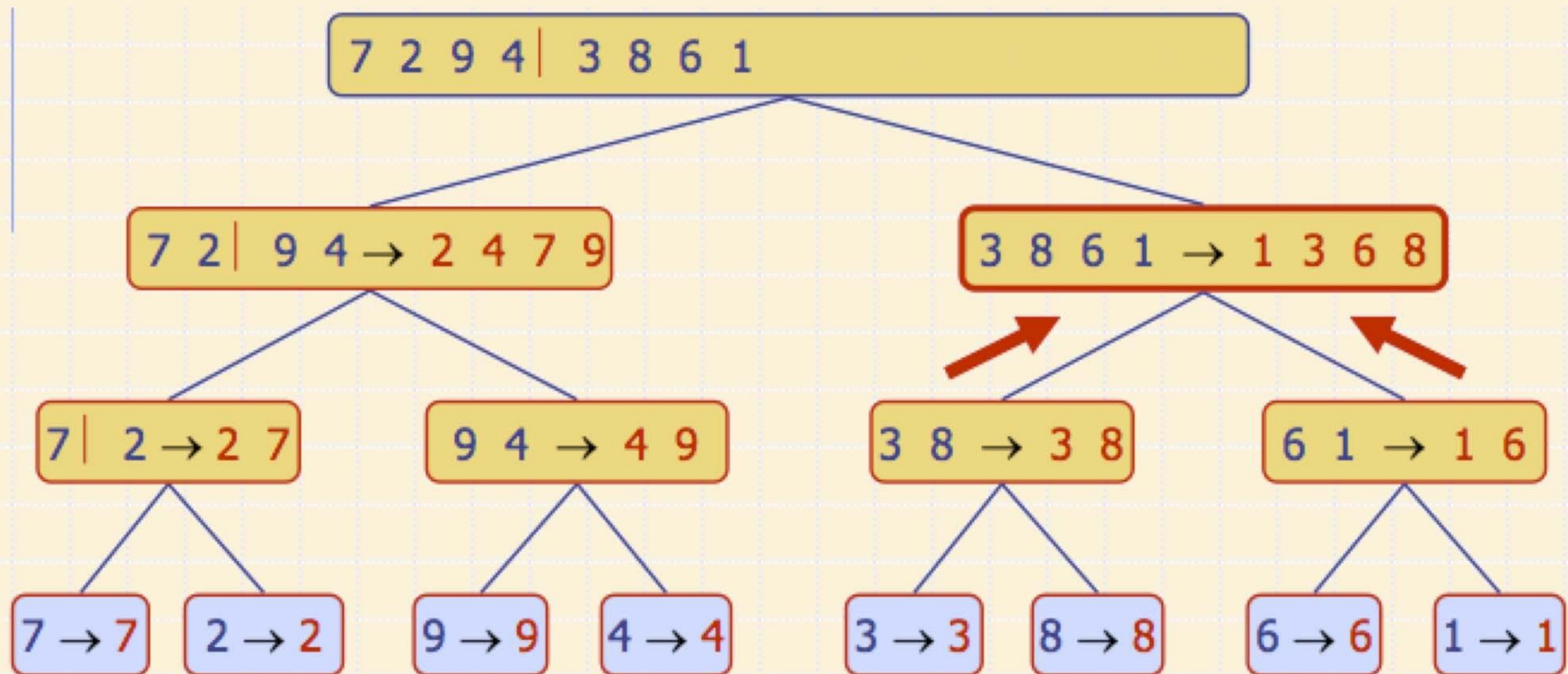
Execution Example (cont.)

➤ Merge



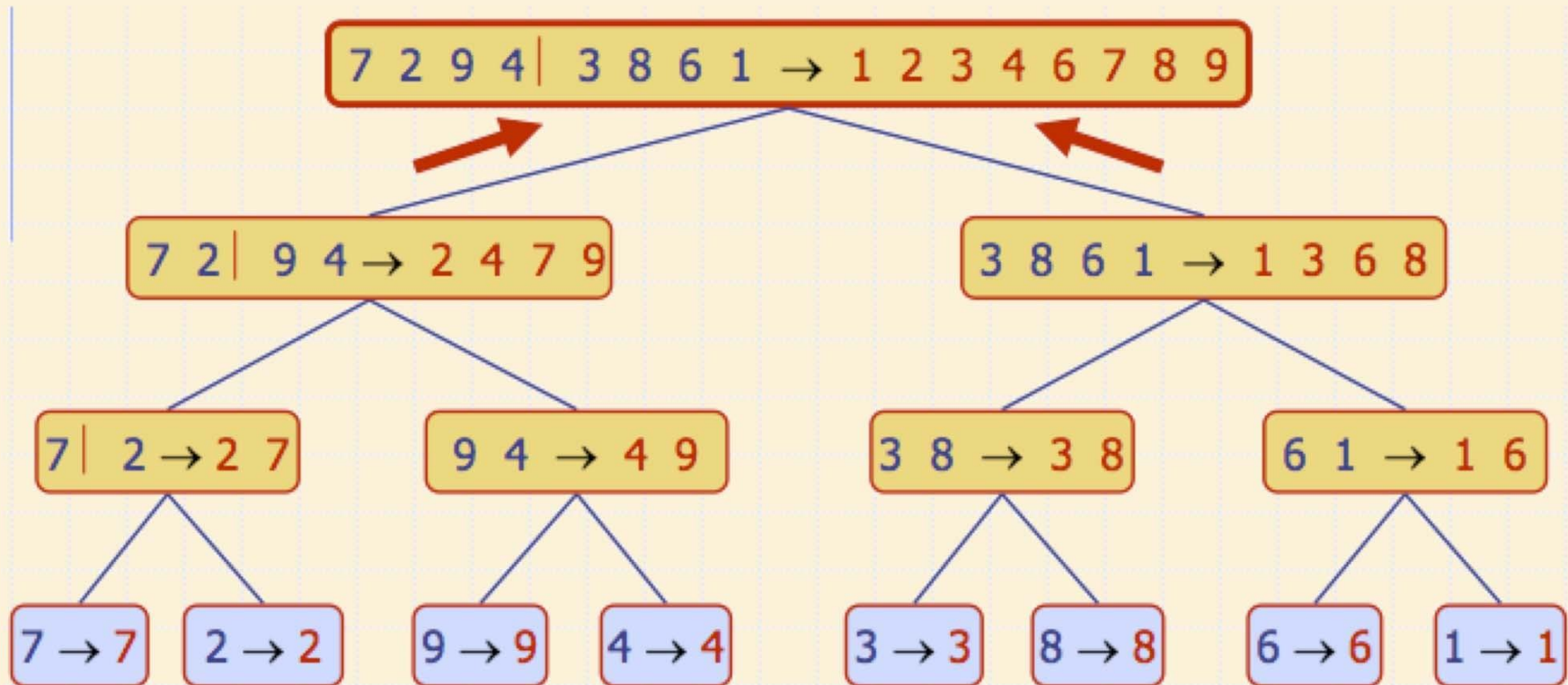
Execution Example (cont.)

- Recursive call, ..., merge, merge



Execution Example (cont.)

➤ Merge



Analysis of Merge-Sort

- The height h of the merge-sort tree is $O(\log n)$
 - ❑ at each recursive call we divide in half the sequence,
- The overall amount of work done at the nodes of depth i is $O(n)$
 - ❑ we partition and merge 2^i sequences of size $n/2^i$
 - ❑ we make 2^{i+1} recursive calls
- Thus, the total running time of merge-sort is $O(n \log n)$

$$T(n) = 2T(n/2) + O(n)$$

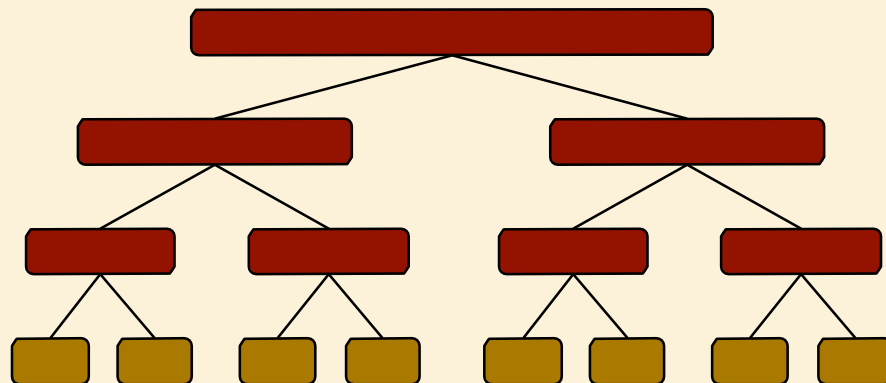
depth	#seqs	size
-------	-------	------

0	1	n
---	---	-----

1	2	$n/2$
---	---	-------

i	2^i	$n/2^i$
-----	-------	---------

...	...	...
-----	-----	-----

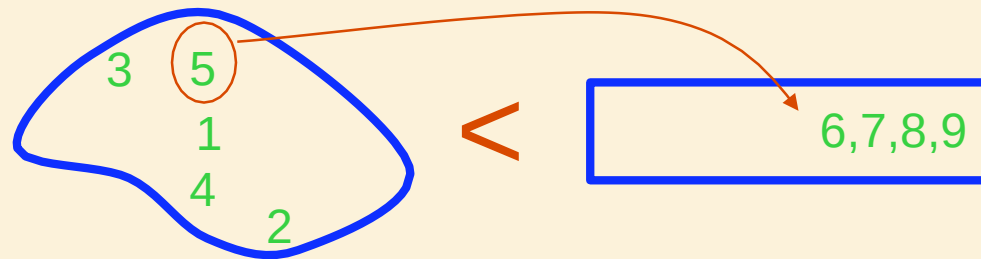


Heapsort

- Invented by Williams & Floyd in 1964
- $O(n \log n)$ worst case – like merge sort
- Sorts in place – like insertion sort
- Combines the best of both algorithms

Selection Sort

Largest i values are sorted on the right.
Remaining values are off to the left.



Max is easier to find if a heap.

Heap-Sort Algorithm

- Build an array-based (max) heap
- Iteratively call `removeMax()` to extract the keys in descending order
- Store the keys as they are extracted in the unused tail portion of the array

Heap-Sort Algorithm

Algorithm HeapSort(S)

Input: S, an unsorted array of comparable elements

Output: S, a sorted array of comparable elements

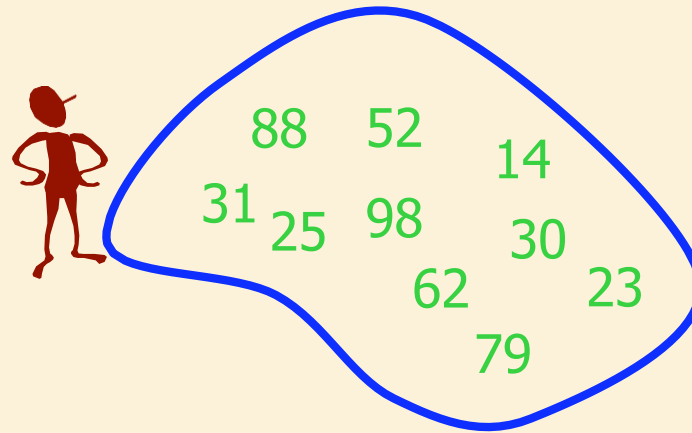
T = MakeMaxHeap (S)

for i = n-1 downto 1

 S[i] = T.removeMax()

Heap Sort Example

Quick-Sort

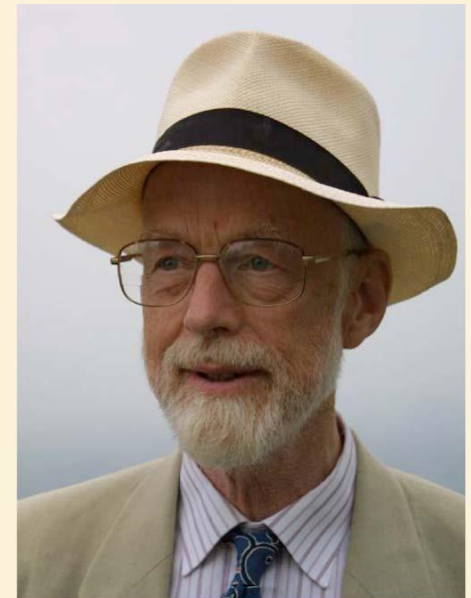


Divide and Conquer



QuickSort

- Invented by C.A.R. Hoare in 1960
- “There are two ways of constructing a software design: One way is to make it so simple that there are obviously no deficiencies, and the other way is to make it so complicated that there are no obvious deficiencies. The first method is far more difficult.”



Quick-Sort

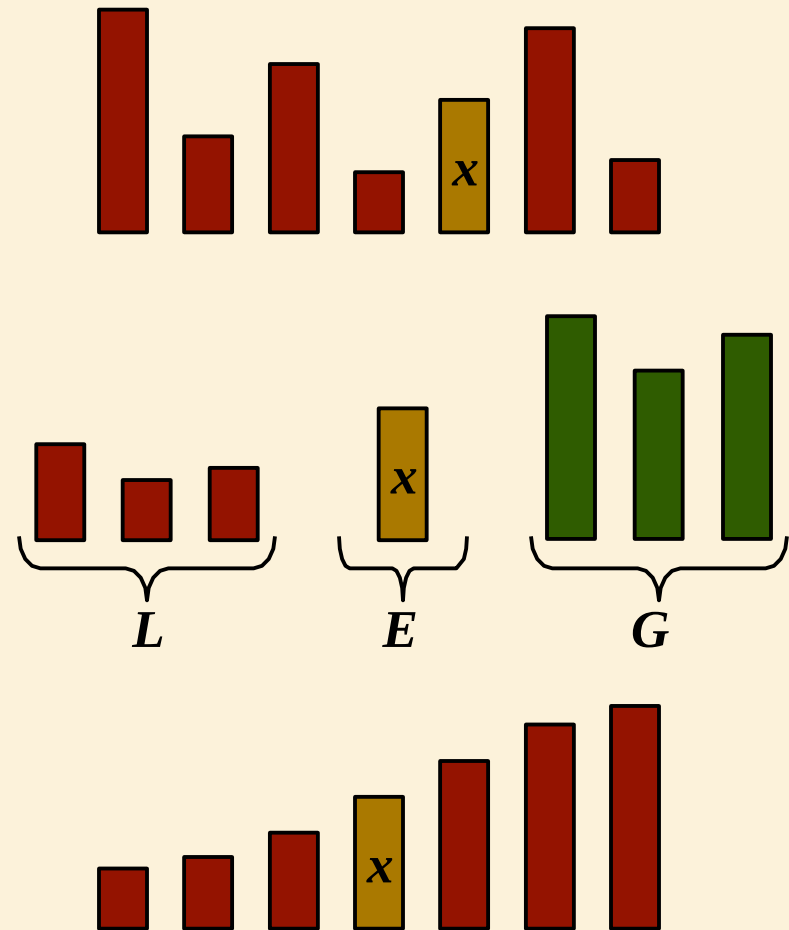
➤ **Quick-sort** is a divide-and-conquer algorithm:

❑ **Divide**: pick a random element x (called a **pivot**) and partition S into

- ✧ L elements less than x
- ✧ E elements equal to x
- ✧ G elements greater than x

❑ **Recur**: Quick-sort L and G

❑ **Conquer**: join L , E and G



The Quick-Sort Algorithm

Algorithm QuickSort(S)

if $S.size() > 1$

$(L, E, G) = \text{Partition}(S)$

 QuickSort(L)

 QuickSort(G)

$S = (L, E, G)$

Partition

- Remove, in turn, each element y from S and
- Insert y into sequence L , E or G , depending on the result of the comparison with the pivot x (e.g., last element in S)
- Each insertion and removal is at the beginning or at the end of a sequence, and hence takes $O(1)$ time
- Thus, partitioning takes $O(n)$ time

Algorithm *Partition*(S)

Input sequence S

Output subsequences L , E , G of the elements of S less than, equal to, or greater than the pivot, resp.

L , E , $G \leftarrow$ empty sequences

$x \leftarrow S.getLast().element$

while $\neg S.isEmpty()$

$y \leftarrow S.removeFirst(S)$

if $y < x$

$L.addLast(y)$

else if $y = x$

$E.addLast(y)$

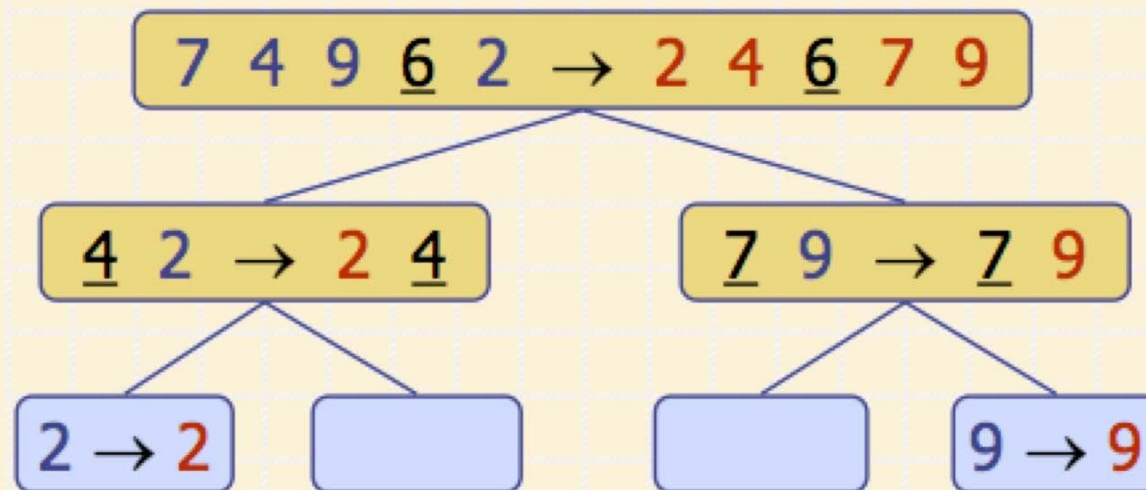
else $\{ y > x \}$

$G.addLast(y)$

return L , E , G

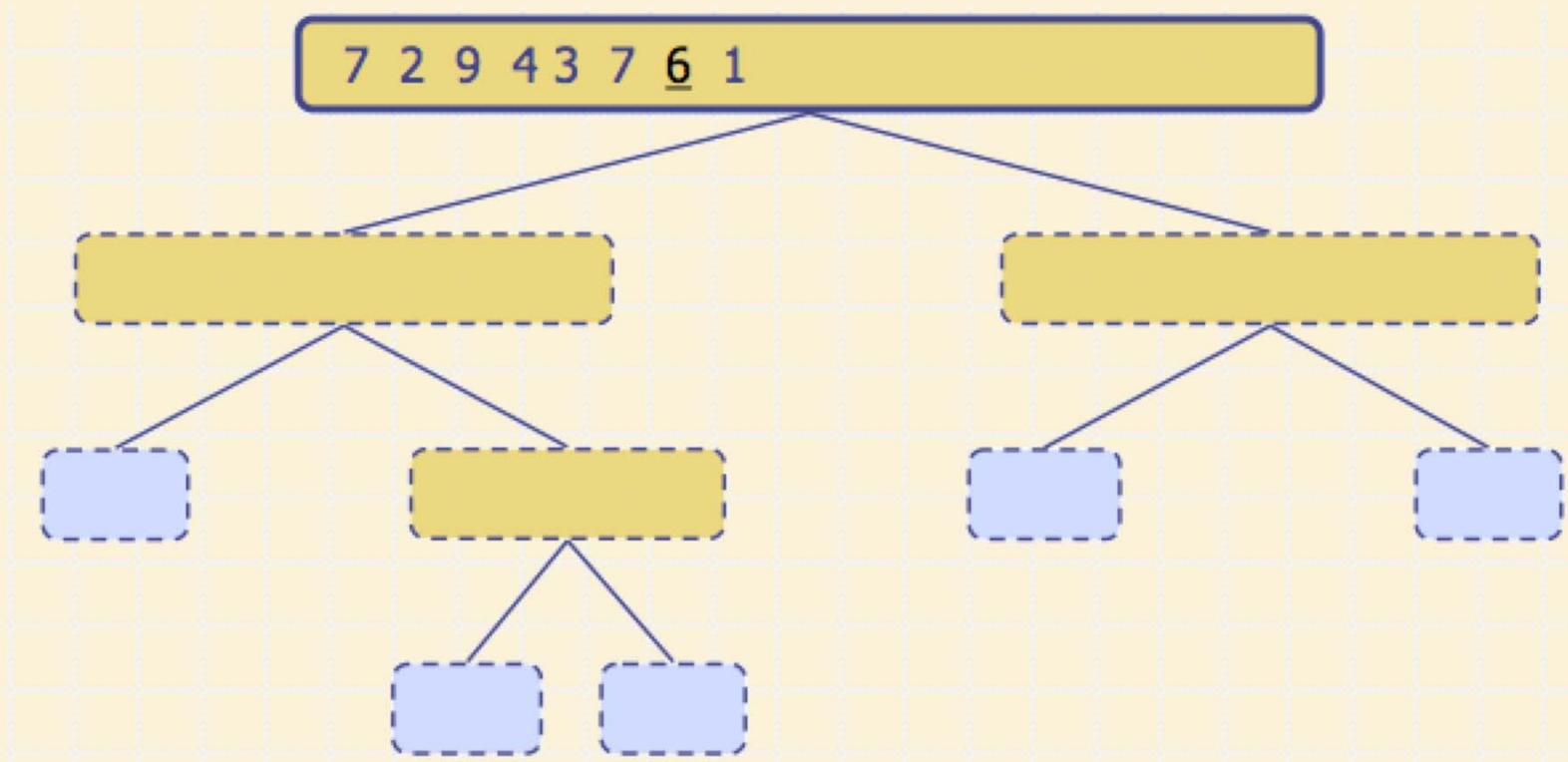
Quick-Sort Tree

- An execution of quick-sort is depicted by a binary tree
 - ❑ Each node represents a recursive call of quick-sort and stores
 - ✧ Unsorted sequence before the execution and its pivot
 - ✧ Sorted sequence at the end of the execution
 - ❑ The root is the initial call
 - ❑ The leaves are calls on subsequences of size 0 or 1



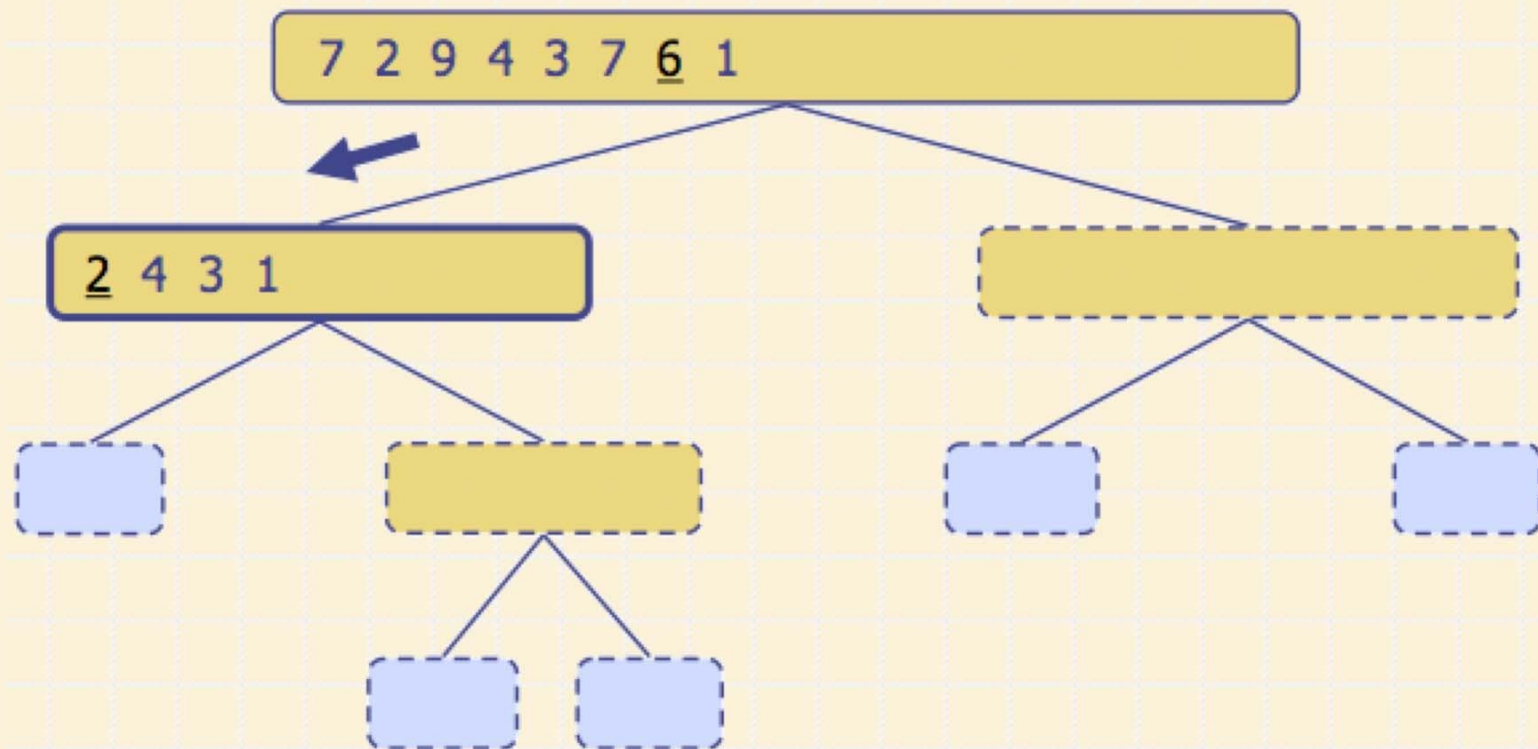
Execution Example

➤ Pivot selection



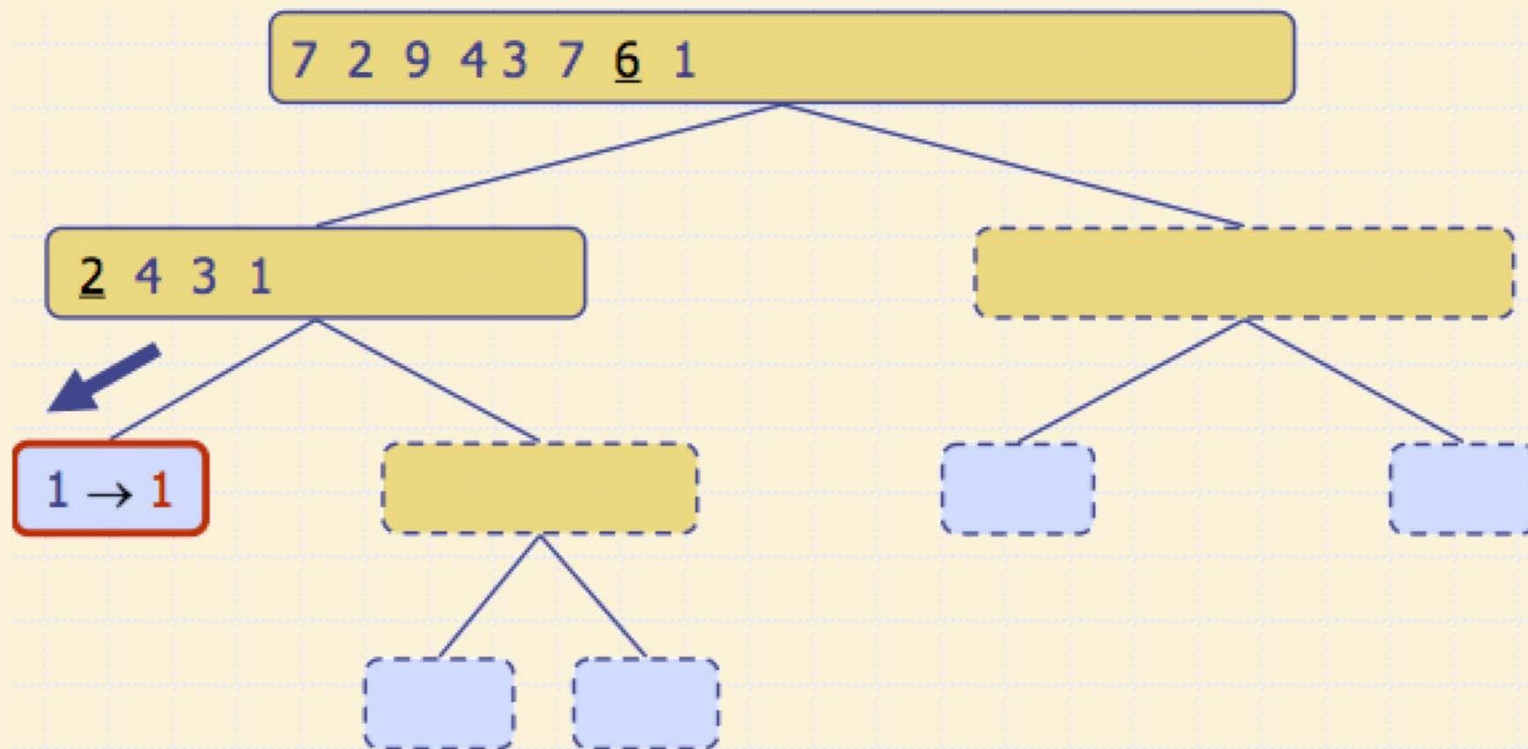
Execution Example (cont.)

- Partition, recursive call, pivot selection



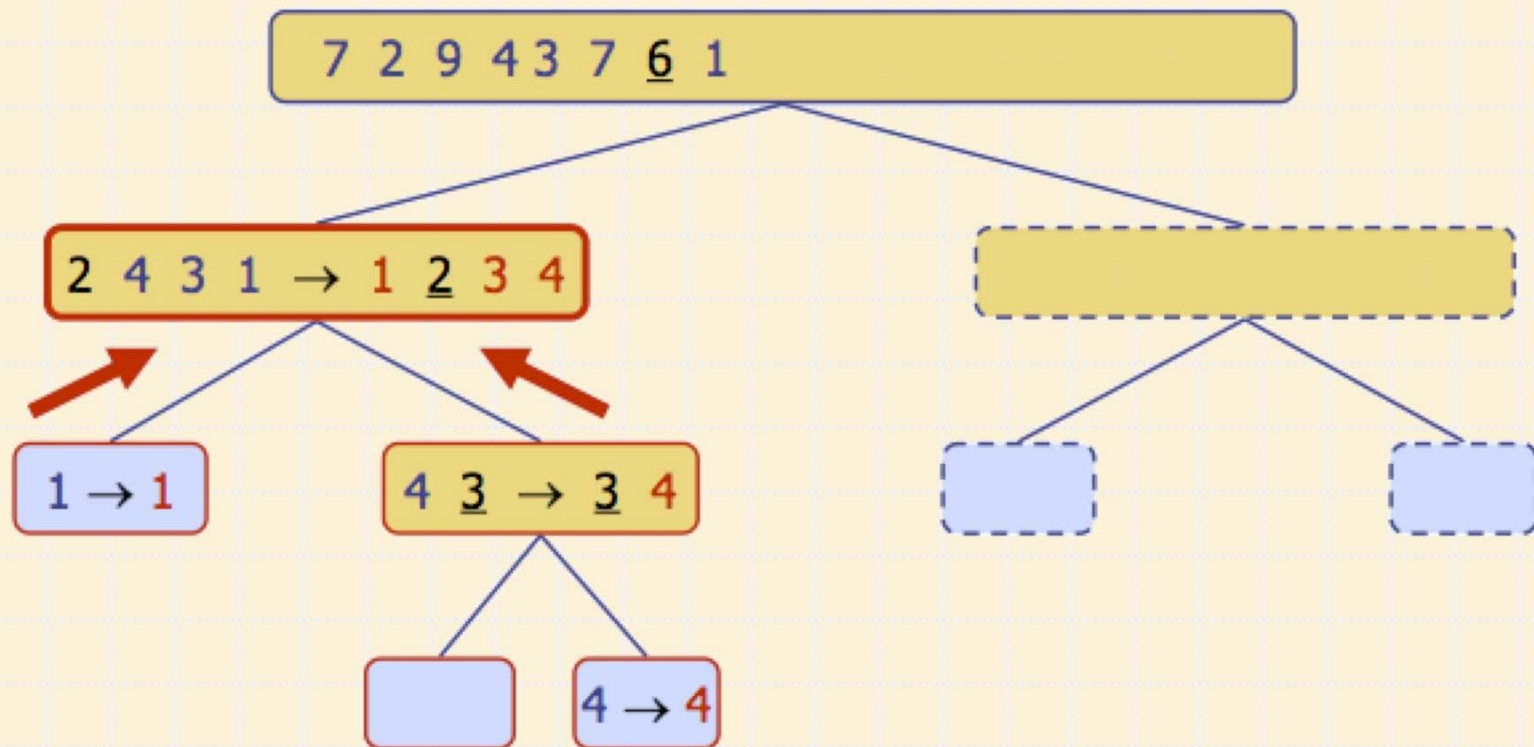
Execution Example (cont.)

- Partition, recursive call, base case



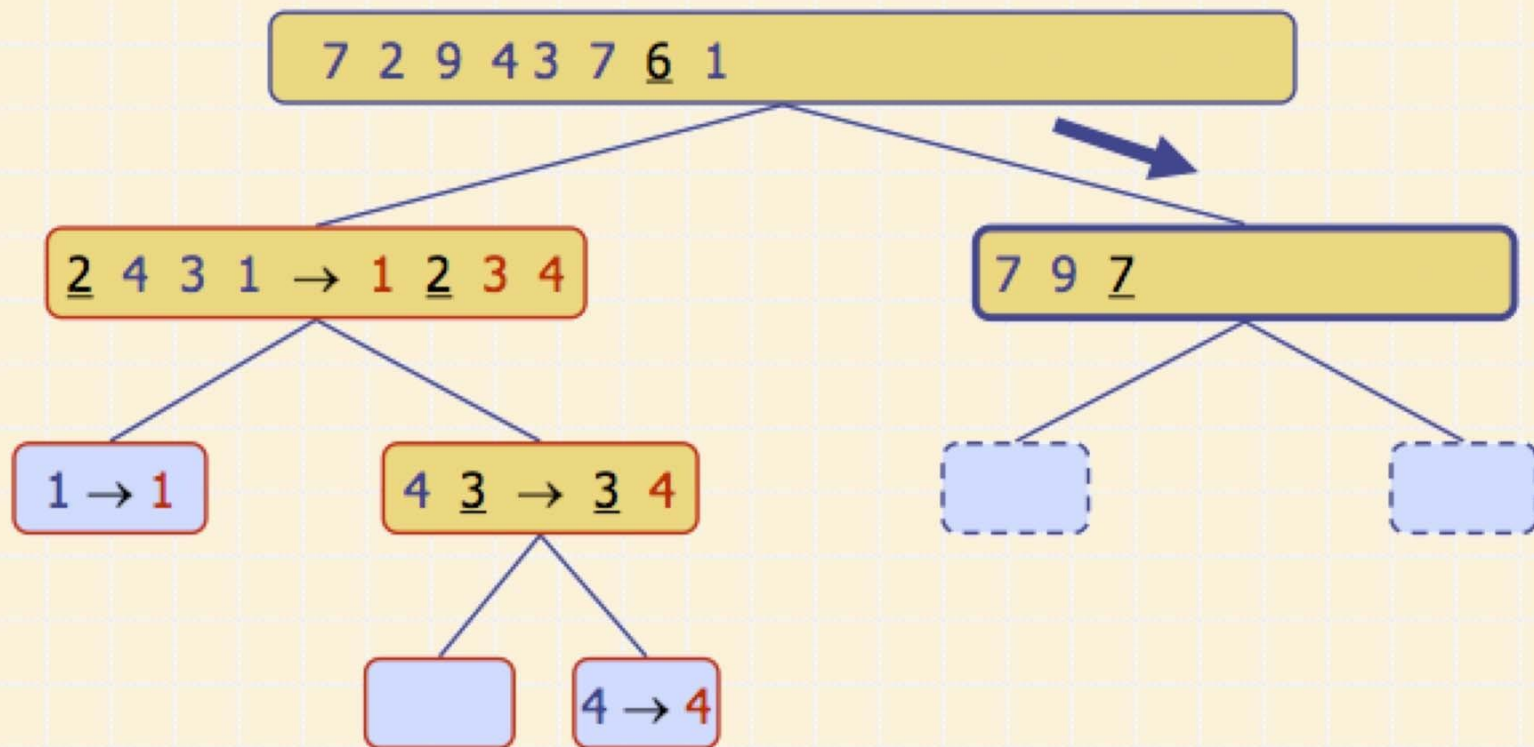
Execution Example (cont.)

- Recursive call, ..., base case, join



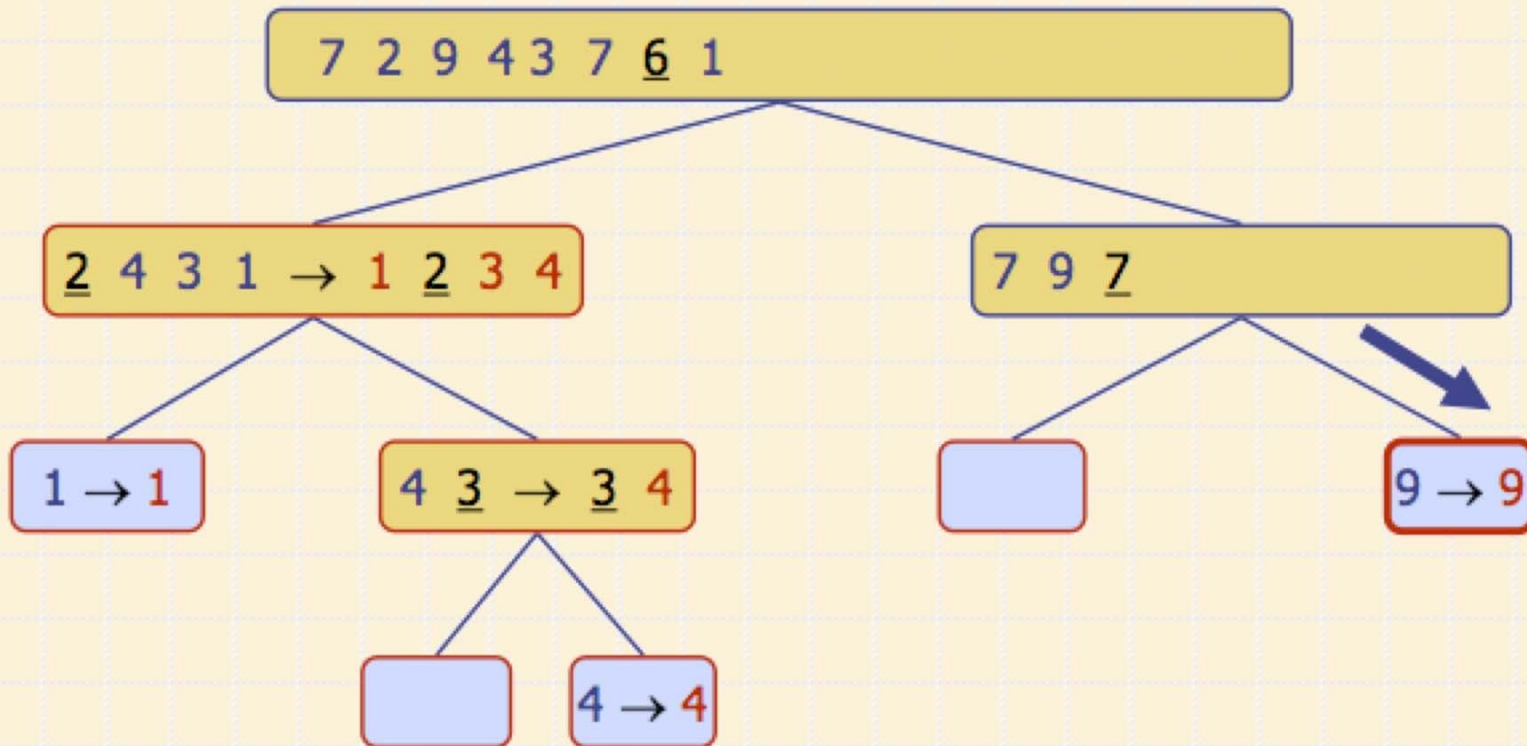
Execution Example (cont.)

- Recursive call, pivot selection



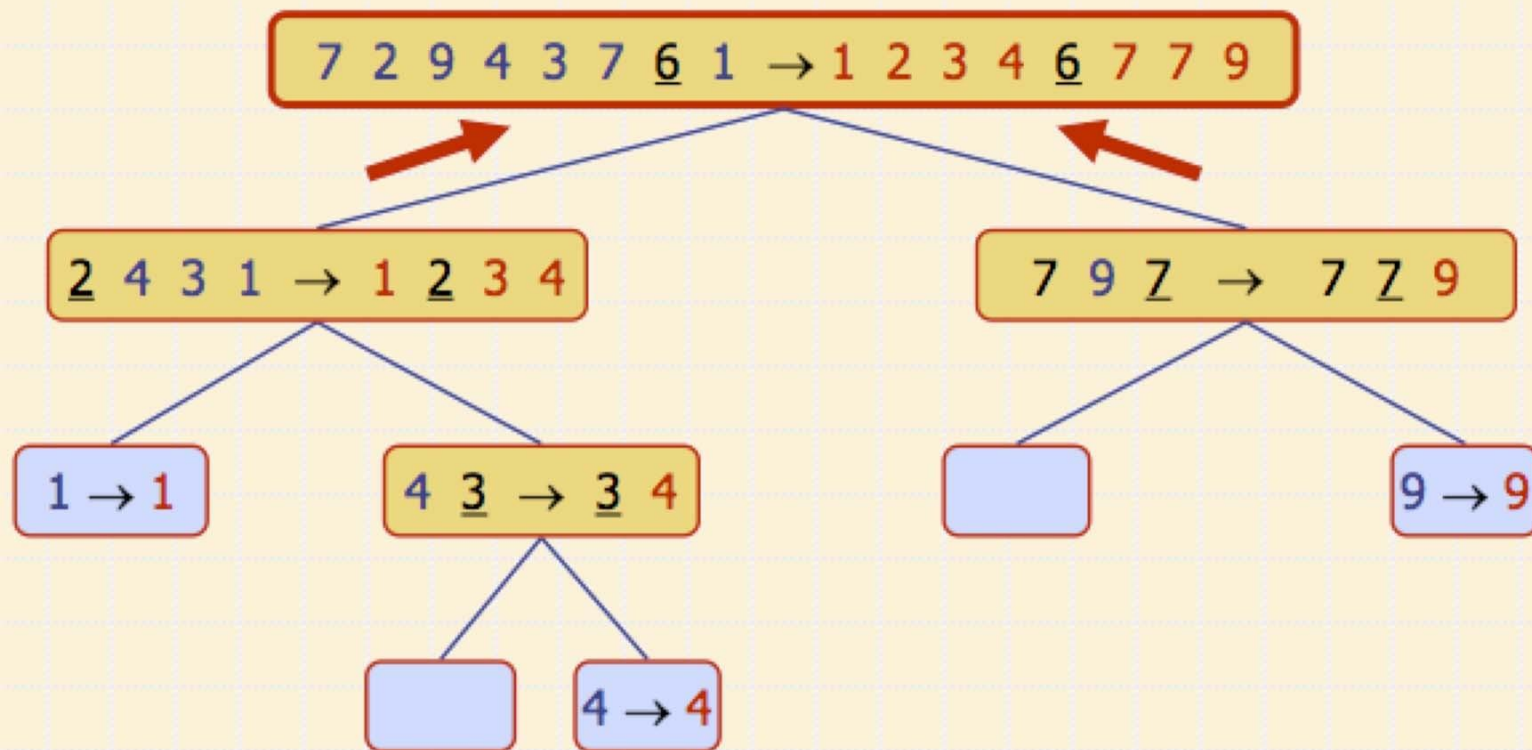
Execution Example (cont.)

- Partition, ..., recursive call, base case



Execution Example (cont.)

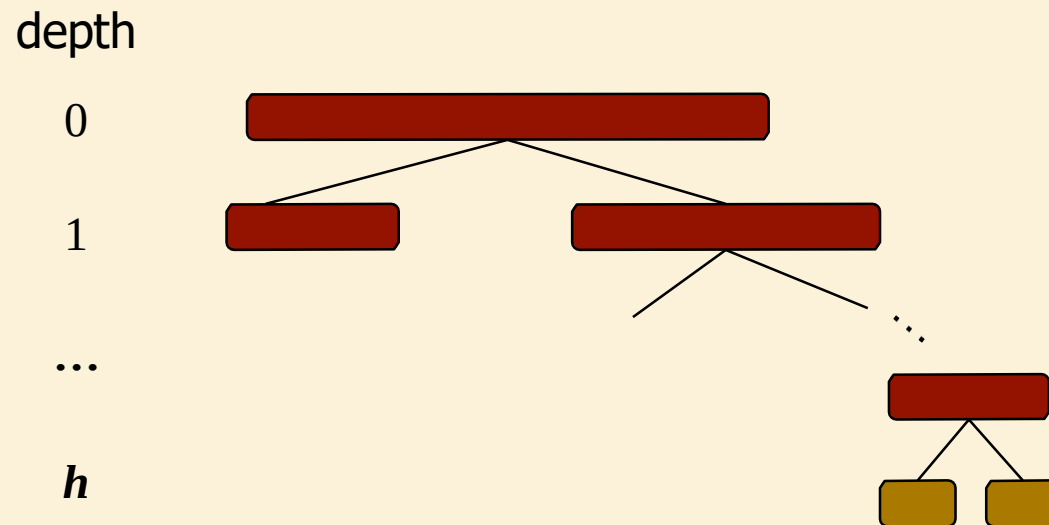
➤ Join, join



Running Time of Quick-Sort

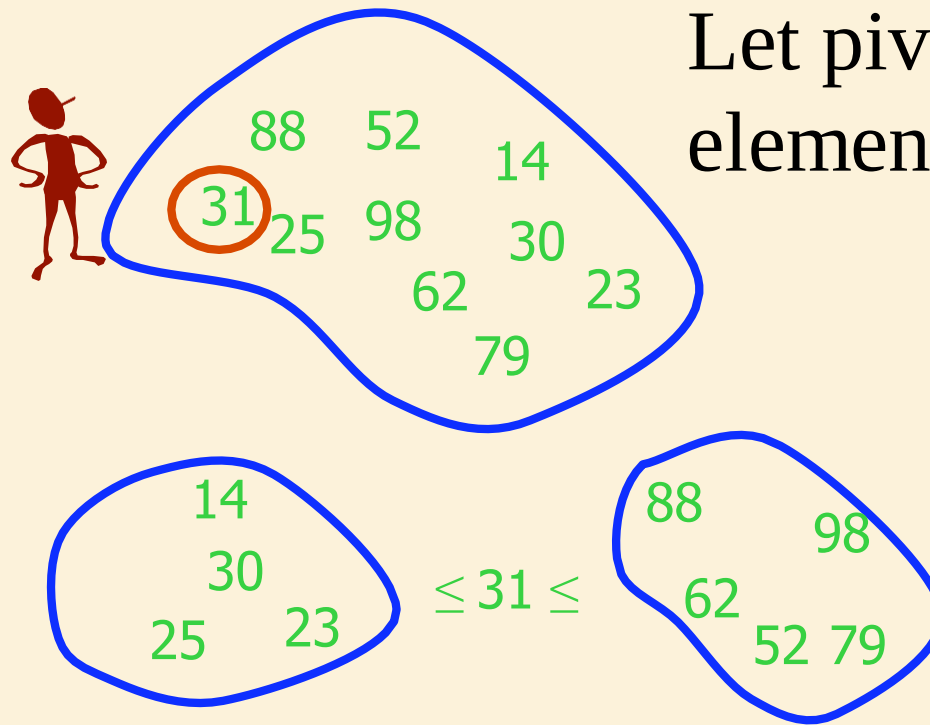
Quick-Sort Running Time

- We can analyze the running time of Quick-Sort using a recursion tree.
- At depth i of the tree, the problem is partitioned into 2^i sub-problems.
- The running time will be determined by how balanced these partitions are.



Quick Sort

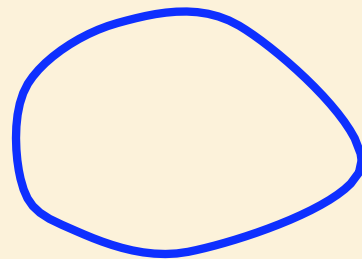
Let pivot be the first element in the list?



Quick Sort



14, 23, 25, 30, 31, 52, 62, 79, 88, 98



$\leq 14 \leq$

23, 25, 30, 31, 52, 62, 79, 88, 98

If the list is already sorted,
then the list is worst case unbalanced.

QuickSort: Choosing the Pivot

➤ Common choices are:

- ☐ random element
- ☐ middle element
- ☐ median of first, middle and last element

Best-Case Running Time

- The best case for quick-sort occurs when each pivot partitions the array in half.
- Then there are $O(\log n)$ levels
- There is $O(n)$ work at each level
- Thus total running time is $O(n \log n)$

depth time

0 n

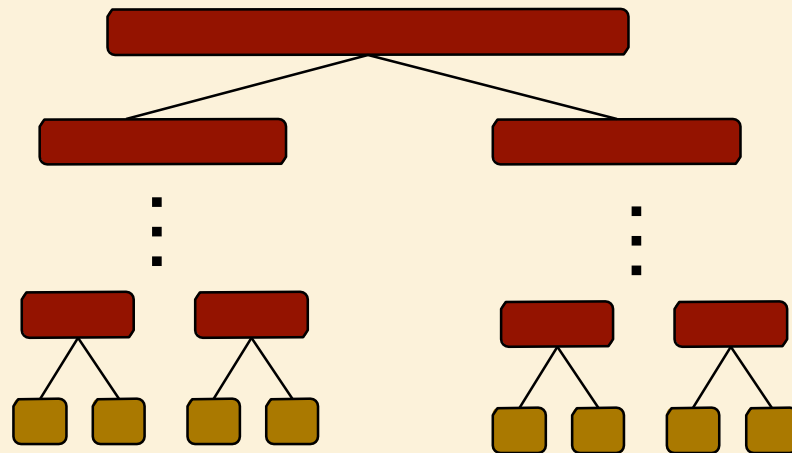
1 n

... ...

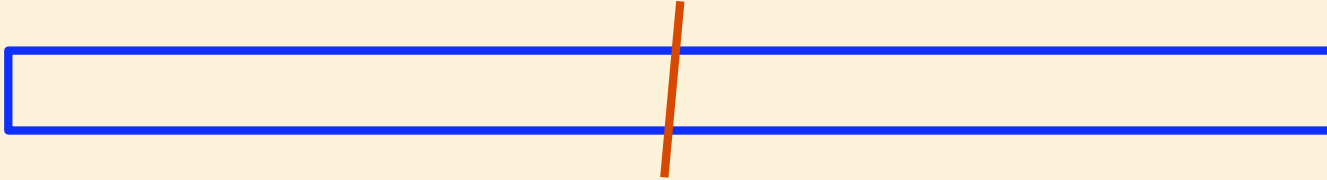
i n

... ...

$\log n$ n



Quick Sort



Best Time: $T(n) = 2T(n/2) + \mathcal{O}(n)$
 $= \mathcal{O}(n \log n)$

Worst Time:

Expected Time:

Properties of QuickSort

➤ Fast?

- ❑ Depends.
- ❑ Worst Case: $\Theta(n^2)$
- ❑ Expected Case: $\Theta(n \log n)$, with small constants

Summary of Comparison Sorts

Algorithm	Best Case	Worst Case	Average Case	In Place	Stable	Comments
Selection	n^2	n^2		Yes	Yes	
Bubble	n	n^2		Yes	Yes	
Insertion	n	n^2		Yes	Yes	Good if often almost sorted
Merge	$n \log n$	$n \log n$		No	Yes	Good for very large datasets that require swapping to disk
Heap	$n \log n$	$n \log n$		Yes	No	Best if guaranteed $n \log n$ required
Quick	$n \log n$	n^2	$n \log n$	Yes	No	Usually fastest in practice

Linear Sorts?

Comparison sorts are very general, but are $\Omega(n \log n)$

Faster sorting may be possible if we can constrain the nature of the input.

Example 1. Counting Sort

- Invented by Harold Seward in 1954.
- **Counting Sort** applies when the elements to be sorted come from a **finite** (and preferably small) **set**.
- For example, the elements to be sorted are integers in the range $[0 \dots k-1]$, for some fixed integer k .
- We can then create an array $V[0 \dots k-1]$ and use it to count the number of elements with each value $[0 \dots k-1]$.
- Then each input element can be placed in exactly the right place in the output array in constant time.

Counting Sort

Input:	1	0	0	1	3	1	1	3	1	0	2	1	0	1	1	2	2	1	0
Output:	0	0	0	0	0	1	1	1	1	1	1	1	1	1	2	2	3	3	3

- Input: N records with integer keys between $[0 \dots 3]$.
- Output: sorted keys.
- Algorithm:
 - ❑ Count frequency of each key value to determine transition locations
 - ❑ Go through the records in order putting them where they go.

CountingSort

Input:	1	0	0	1	3	1	1	3	1	0	2	1	0	1	1	2	2	1	0
Output:	0	0	0	0	0	1	1	1	1	1	1	1	1	1	2	2	2	3	3
Index:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

Stable sort: If two keys are the same, their order does not change.

Thus the 4th record in input with digit 1 must be
the 4th record in output with digit 1.

It belongs at output index 8, because 8 records go before it
ie, 5 records with a smaller digit & 3 records with the same digit

Count These!

CountingSort

Input:	1	0	0	1	3	1	1	3	1	0	2	1	0	1	1	2	2	1	0
Output:																			
Index:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

Value v:	0	1	2	3
# of records with digit v:	5	9	3	2

N records. Time to count? $\theta(N)$

CountingSort

Input:	1	0	0	1	3	1	1	3	1	0	2	1	0	1	1	2	2	1	0
Output:																			
Index:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

Value v:	0	1	2	3
# of records with digit v:	5	9	3	3
# of records with digit < v:	0	5	14	17

N records, k different values. Time to count? $\theta(k)$

CountingSort

Input:	1	0	0	1	3	1	1	3	1	0	2	1	0	1	1	2	2	1	0
Output:	0	0	0	0	0	1	1	1	1	1	1	1	1	1	2	2	2	3	3
Index:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

Value v:

0	1	2	3
0	5	14	17

of records with digit $< v$:

= location of first record with digit v.

CountingSort

Input:	1	0	0	1	3	1	1	3	1	0	2	1	0	1	1	2	2	1	0
Output:	0	?				1													
Index:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

Value v:

0	1	2	3
0	5	14	17

Location of first record
with digit v.

Algorithm: Go through the records in order
putting them where they go.

Loop Invariant

- The first $i-1$ keys have been placed in the correct locations in the output array
- The auxiliary data structure v indicates the location at which to place the i^{th} key for each possible key value from $[1..k-1]$.

CountingSort

Input:	1	0	0	1	3	1	1	3	1	0	2	1	0	1	1	2	2	1	0
Output:						1													
Index:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

Value v:

0	1	2	3
0	5	14	17

Location of **next** record
with digit v.

Algorithm: Go through the records in order
putting them where they go.

CountingSort

Input:	1	0	0	1	3	1	1	3	1	0	2	1	0	1	1	2	2	1	0
Output:	0					1													
Index:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

Value v:

0	1	2	3
0	6	14	17

Location of **next** record
with digit v.

Algorithm: Go through the records in order
putting them where they go.

CountingSort

Input:	1	0	0	1	3	1	1	3	1	0	2	1	0	1	1	2	2	1	0
Output:	0	0				1													
Index:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

Value v:

0	1	2	3
1	6	14	17

Location of **next** record
with digit v.

Algorithm: Go through the records in order
putting them where they go.

CountingSort

Input:	1	0	0	1	3	1	1	3	1	0	2	1	0	1	1	2	2	1	0
Output:	0	0				1	1												
Index:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

Value v:

0	1	2	3
2	6	14	17

Location of **next** record
with digit v.

Algorithm: Go through the records in order
putting them where they go.

CountingSort

Input:	1	0	0	1	3	1	1	3	1	0	2	1	0	1	1	2	2	1	0
Output:	0	0				1	1											3	
Index:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

Value v:

0	1	2	3
2	7	14	17

Location of **next** record
with digit v.

Algorithm: Go through the records in order
putting them where they go.

CountingSort

Input:	1	0	0	1	3	1	1	3	1	0	2	1	0	1	1	2	2	1	0
Output:	0	0				1	1	1										3	
Index:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

Value v:

0	1	2	3
2	7	14	18

Location of **next** record
with digit v.

Algorithm: Go through the records in order
putting them where they go.

CountingSort

Input:	1	0	0	1	3	1	1	3	1	0	2	1	0	1	1	2	2	1	0
Output:	0	0				1	1	1	1									3	
Index:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18



Value v:

0	1	2	3
2	8	14	18

Location of **next** record
with digit v.

Algorithm: Go through the records in order
putting them where they go.

CountingSort

Input:	1	0	0	1	3	1	1	3	1	0	2	1	0	1	1	2	2	1	0
Output:	0	0				1	1	1	1									3	3
Index:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

Value v:

0	1	2	3
2	9	14	18

Location of **next** record
with digit v.

Algorithm: Go through the records in order
putting them where they go.

CountingSort

Input:	1	0	0	1	3	1	1	3	1	0	2	1	0	1	1	2	2	1	0
Output:	0	0				1	1	1	1	1								3	3
Index:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

Value v:

0	1	2	3
2	9	14	19

Location of **next** record
with digit v.

Algorithm: Go through the records in order
putting them where they go.

CountingSort

Input:	1	0	0	1	3	1	1	3	1	0	2	1	0	1	1	2	2	1	0
Output:	0	0	0			1	1	1	1	1								3	3
Index:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

Value v:

0	1	2	3
2	10	14	19

Location of **next** record
with digit v.

Algorithm: Go through the records in order
putting them where they go.

CountingSort

Input:	1	0	0	1	3	1	1	3	1	0	2	1	0	1	1	2	2	1	0
Output:	0	0	0			1	1	1	1	1					2			3	3
Index:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

Value v:

0	1	2	3
3	10	14	19

Location of **next** record
with digit v.

Algorithm: Go through the records in order
putting them where they go.

CountingSort

Input:	1	0	0	1	3	1	1	3	1	0	2	1	0	1	1	2	2	1	0
Output:	0	0	0			1	1	1	1	1	1				2			3	3
Index:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

Value v:

0	1	2	3
3	10	15	19

Location of **next** record
with digit v.

Algorithm: Go through the records in order
putting them where they go.

CountingSort

Input:	1	0	0	1	3	1	1	3	1	0	2	1	0	1	1	2	2	1	0
Output:	0	0	0			1	1	1	1	1	1				2			3	3
Index:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

Value v:

0	1	2	3
3	10	15	19

Location of **next** record
with digit v.

Algorithm: Go through the records in order
putting them where they go.

CountingSort

Input:	1	0	0	1	3	1	1	3	1	0	2	1	0	1	1	2	2	1	0
Output:	0	0	0	0	0	1	1	1	1	1	1	1	1	1	2	2	2	3	3
Index:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

Value v:

0	1	2	3
5	14	17	19

Location of **next** record
with digit v.

Time = $\theta(N)$

Total = $\theta(N+k)$

Example 2. RadixSort

Input:

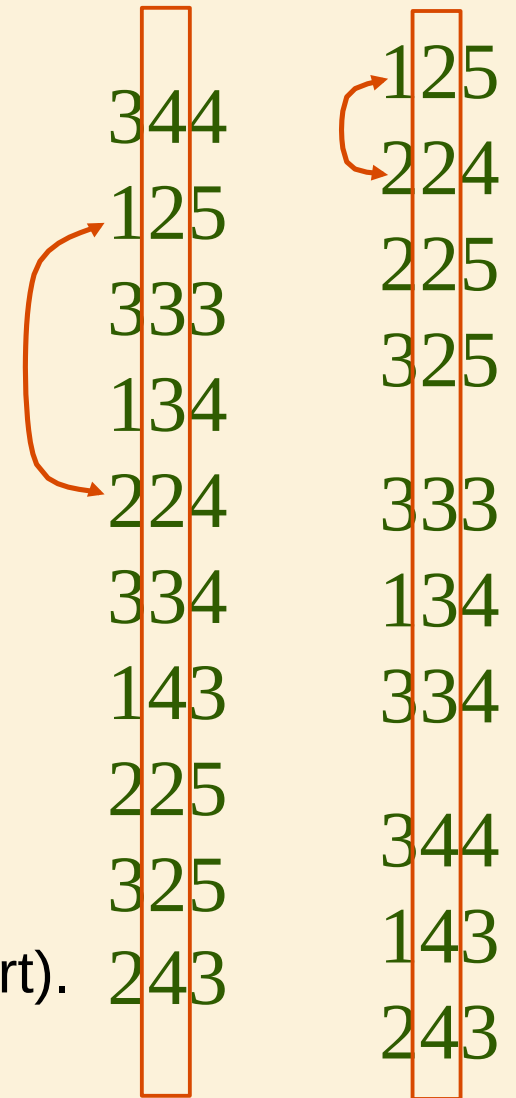
- An array of N numbers.
- Each number contains d digits.
- Each digit between $[0 \dots k-1]$

Output:

- Sorted numbers.

Digit Sort:

- Select one digit
- Separate numbers into k piles based on selected digit (e.g., Counting Sort).



Stable sort: If two cards are the same for that digit, their order does not change.

RadixSort

344
125
333
134
224
334
143
225
325
243

Sort wrt which
digit first?

The most
significant.

125
134
143
224
225
243
344
333
334
325

Sort wrt which
digit Second?

The next most
significant.

125
224
225
325
134
333
334
143
243
344

All meaning in first sort lost.

RadixSort

344
125
333
134
224
334
143
225
325
243

Sort wrt which
digit first?

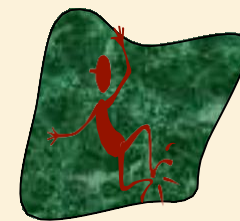
The least
significant.

333
143
243
344
134
224
334
125
225
325

Sort wrt which
digit Second?

The next least
significant.

224
125
225
325
333
134
334
143
243
344



RadixSort

344

125

333

134

224

334

143

225

325

243

Sort wrt which
digit first?

The least
significant.

333

143

243

344

134

224

334

125

225

325

Sort wrt which
digit Second?

The next least
significant.

2 24

1 25

2 25

3 25

3 33

1 34

3 34

1 43

2 43

3 44

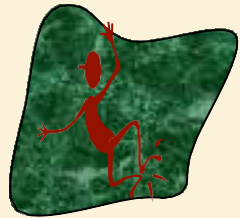


Is sorted wrt least sig. 2 digits.



RadixSort

2	24
1	25
2	25
3	25
3	33
1	34
3	34
1	43
2	43
3	44



Is sorted wrt
first i digits.



Sort wrt $i+1$ st
digit.

1	25
1	34
1	43
2	24
2	25
2	43
3	25
3	33
3	34
3	44



Is sorted wrt
first $i+1$ digits.

These are in the
correct order
because sorted
wrt high order digit

RadixSort

2 24

1 25

2 25

3 25

3 33

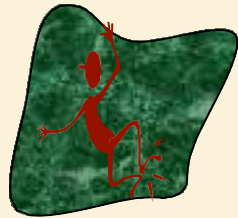
1 34

3 34

1 43

2 43

3 44
i+1



Is sorted wrt
first i digits.



Sort wrt $i+1$ st
digit.

1 25

1 34

1 43

2 24

2 25

2 43

3 25

3 33

3 34

3 44



Is sorted wrt
first $i+1$ digits.

These are in the
correct order
because was sorted &
stable sort left sorted

Loop Invariant



- The keys have been correctly stable-sorted with respect to the $i-1$ least-significant digits.

Running Time

RADIX-SORT(A, d)

for $i \leftarrow 1$ to d

do use a stable sort to sort array A on digit i

Running time is $\Theta(d(n + k))$

Where

$d = \#$ of digits in each number

$n = \#$ of elements to be sorted

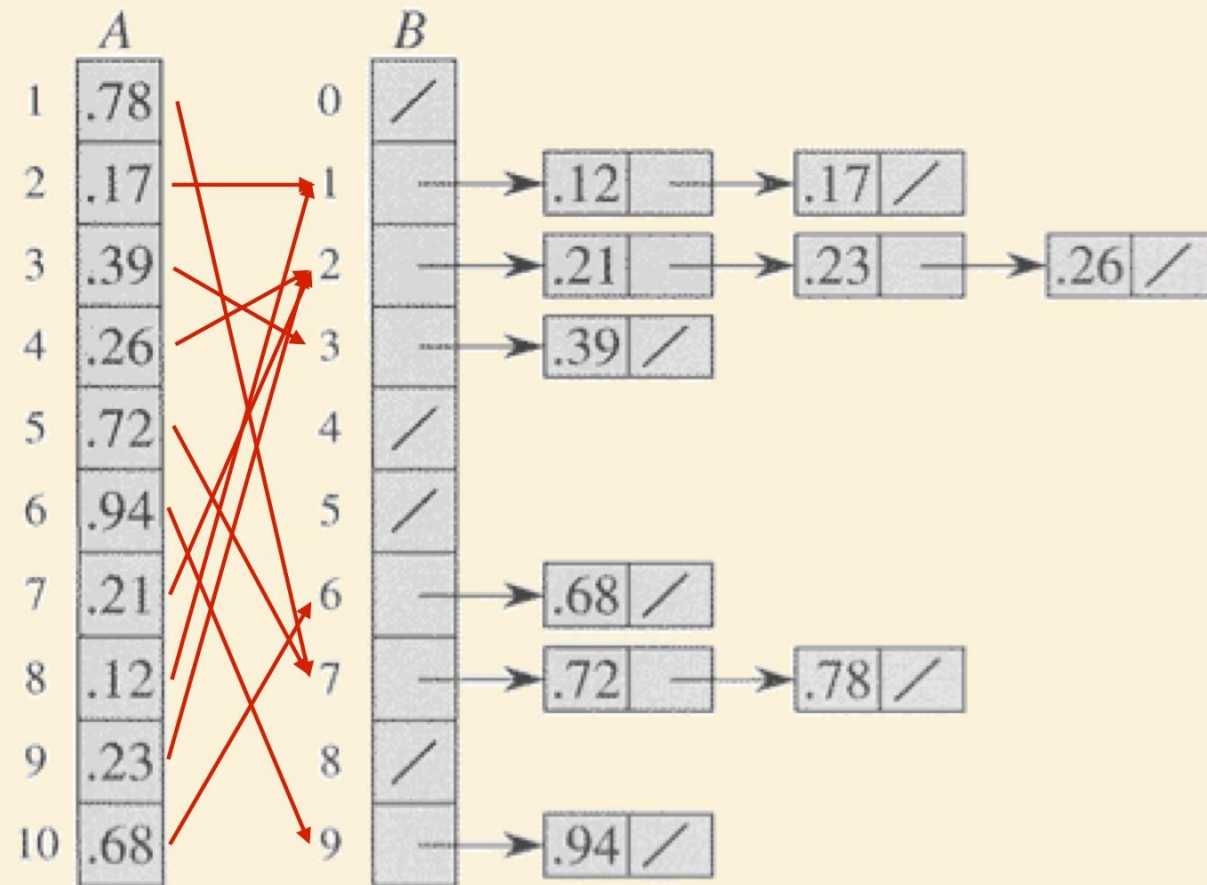
$k = \#$ of possible values for each digit

Example 3. Bucket Sort

- Applicable if input is constrained to finite interval, e.g., $[0 \dots 1)$.
- If input is random and uniformly distributed, **expected** run time is $\Theta(n)$.

Bucket Sort

insert $A[i]$ into list $B[\lfloor n \cdot A[i] \rfloor]$



Loop Invariants



➤ Loop 1

- ❑ The first $i-1$ keys have been correctly placed into buckets of width $1/n$.

➤ Loop 2

- ❑ The keys **within** each of the first $i-1$ buckets have been correctly stable-sorted.

PseudoCode

BUCKET-SORT(A, n)

Expected Running Time

for $i \leftarrow 1$ **to** n

do insert $A[i]$ into list $B[\lfloor n \cdot A[i] \rfloor]$ $\leftarrow \Theta(1) \times n$

for $i \leftarrow 0$ **to** $n - 1$

do sort list $B[i]$ with insertion sort $\leftarrow \Theta(1) \times n$

concatenate lists $B[0], B[1], \dots, B[n - 1]$ $\leftarrow \Theta(n)$

return the concatenated lists

$\Theta(n)$